

SVEUČILIŠTE JOSIPA JURJA STROSSMAYERA U OSIJEKU

**FAKULTET ELEKTROTEHNIKE, RAČUNARSTVA I
INFORMACIJSKIH TEHNOLOGIJA OSIJEK**

Sveučilišni diplomski studij računarstva

OBRADA SLIKE I RAČUNALNI VID

**Prepoznavanje kirurških instrumenata na
intraoperativnim videozapisima pomoću YOLOv8**

Akademska godina 2025/2026

Karlo Vlašić

Osijek, 2026

SADRŽAJ

1. UVOD.....	1
2. DETEKCIJA OBJEKATA I YOLO ARHITEKTURA.....	3
2.1. Detekcija objekata.....	3
2.2. Moderni detektori objekata.....	8
2.2.1. YOLOv1.....	10
2.3. Razvoj arhitekture YOLO modela.....	12
2.3.1. YOLOv1.....	12
2.3.2. YOLOv2.....	14
2.3.3. YOLOv3.....	15
2.3.4. YOLOv4.....	17
2.3.5. YOLOv5.....	18
2.3.6. YOLOv6.....	20
2.3.7. YOLOv7.....	21
2.3.8. YOLOv8.....	22
3. SKUP PODATAKA.....	24
4. IMPLEMENTACIJA I TRENIRANJE YOLOv8 MODELA.....	29
5. EVALUACIJA REZULTATA.....	31
6. RASPRAVA I ZAKLJUČAK.....	35

1. UVOD

Jedan od najranijih znakova kirurgije koja je ostavila trag na čovjeku datira oko 6500 godina prije nove ere, a kroz razvoj drugih znanosti čijim je kirurgija okužena, ona je iz pojma straha prerasla u relativno siguran način uklanjanja bolesti iz ljudskog tijela. Problem brzog oporavka, s obzirom na zahvat, postoji i dalje, kao i što postoji mogućnost ljudske greške koji uvelike smanjuje sigurnost zahvata i oporavak od istog.

Jedan od zahvata s bržim oporavkom se naziva minimalno invazivni zahvat, kao što je laparoskopija, a osim bržeg oporavka, smanjuje se gubitak krvi te je rizik od infekcija, također smanjen. Takve metode zahvata su složenije od tradicionalnih, što još više opterećuje kirurški tim koji istovremeno treba upravljati složenim instrumentima, promatrati prikaz operacijskog polja, promatrati stanje pacijenta i druge operacije za uspješno izvršenje operacija.

Kako bi kirurški tim imao pregled na operacijsko polje minimalno invazivnim zahvatima, poput onih zahvata izvedenim da Vinci kirurškim sustavom, koriste se 3D kamere s visokom rezolucijom, endoskopi. Oni su uređaji s kamerom na vrhu fleksibilne cijevi te spojene na ekran kako bi se prikazala slika s kamere.

Iako su kamere visoke rezolucijom, okolina u kojoj se nalazi otežava shvaćanje prikaza. Prisutnost tjelesnih tekućina, kao i refleksija svjetlosti s metalnih površina instrumenata smanjuje vidljivost, brza promjena kuta kamere i pomicanje tkiva te instrumenti koji djelomično ili potpuno nestanu iz vidnog polja kamere otežavaju interpretaciju prikaza operacijskog polja. Budući kako spora interpretacija prikaza operacijskog polja može dovesti do neželjenih situacija tijekom izvođenja zahvata, ona se želi izbjeći, a prepoznavanje kirurških instrumenata u intraoperativnim zahvatima pomoću YOLOv8 modela može biti upotrebjeno u interpretaciji prikaza i ubrzati interpretaciju prikaza.

Ubrzanje interpretacije prikaza dovodi do povećanje sigurnosti pacijenata, praćenje instrumenata može ubrzati tok operacije kako bi se preciznije odredila faza operacije i priprema za sljedeći korak operacije te time ubrzalo vrijeme trajanja operacije, analiza videozapisa može objektivno evaluirati vještinu specijaliziranih i usmjeriti iste na nepreciznost izvršenih pokreta, a može se primijeniti uz robotski sustav koji prilagođava kretanje instrumenata obzirom na određene parametre.

Rezultat primjena metoda prepoznavanja instrumenata tijekom operativnih zahvata dovodi do mnogih poboljšanja, a takve metode u svojoj srži nisu različite od drugih metoda prepoznavanja objekata koje također koriste i primjenjuju algoritme računalnog vida i strojnog učenja za automatsko otkrivanje, klasifikaciju i praćenje objekata. Dok se u medicini prilagođava

već navedenim problemima okoline, srž problema detekcije objekata može se podijeliti na prepoznavanje ili klasifikaciju objekta i označavanje položaja ili lokalizaciju objekta.

Problem klasifikacije može se svesti na određivanje objekta unutar unaprijed definiranog skupa klasa. U slučaju ovog rada, određuje se specifičan instrument od skupa instrumenata, poput Maryland disektora, bipolarnih hvataljki, prograsp hvataljki, iglodržača... Dok se problem lokalizacije može svesti na određivanje prostornog položaja i veličine unutar slike. Izražava se pomoću pravokutnika koji se naziva granični okvir te se zapisuje u obliku koji obuhvaća x i y koordinate i visinu i širinu pravokutnika.

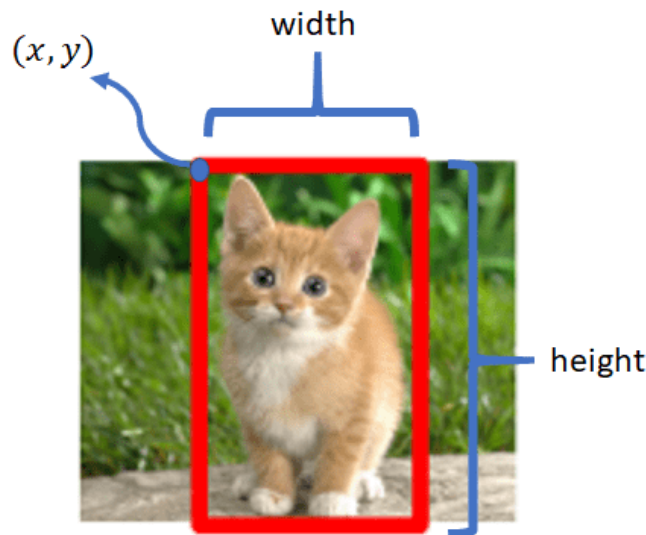
U sljedećim poglavljima detaljnije se prolazi kroz YOLO arhitekturu, skup podataka koji se koristi kako bi se izgradio model, način učenja i učenje modela, primjena modela te evaluacija modela, i posljednje, poboljšanje modela s obzirom na evaluaciju.

2. DETEKCIJA OBJEKATA I YOLO ARHITEKTURA

Detekcija objekata je prirodno proširenje zadatka klasifikacije gdje se za sve objekte od interesa na slici dodijeljuje klasa i lokacija. Takav zadatak je jednostavan za čovjeka, dok za računalo nije toliko jednostavan, čak, izazovan zadatak, sve do 2012. godine, kada se značajno pojednostavnio pojavom dubokih neuronskih mreža.

2.1. Detekcija objekata

Rješavanje problema lokalizacija ili dodijeljivanje lokacije objekta od interesa započinje s definiranjem ishodišta u gornjem lijevom kutu slike, budući kako je slika zapravo matrica s dvije dimenzije, a te dvije dimenzije se mogu izraziti kao x i y . Naravno, u ishodištu imaju iznose od 0, a iznos dimenzije raste pomicanjem na desnu stranu u slučaju x dimenzije, odnosno, pomicanjem dolje u slučaju y dimenzije.



slika 2.1. lokalizacije objekta od interesa

Zapis lokacije objekta od interesa može se prikazati graničnim okvirom koji može biti izražen na nekoliko načina, a oni prikladan zapis se odabire obzirom na njegovu primjenu. Kao što je prije spomenuto, u zapisu (x, y, w, h) , gdje x označava vrijednost x dimenzije i početak graničnog okvira u x dimenziji, y označava vrijednost y dimenzije i početak graničnog okvira u istoj, w širinu okvira ili vrijednost pomaka po x dimenziji te h označava visinu okvira ili vrijednost pomaka po y dimenziji. Interpretacija takvog zapisa može se vidjeti na slici 2.1. Zapis se može pohraniti u obliku $(x_{min}, y_{min}, x_{max}, y_{max})$ gdje je x_{min} lijevi rub ili minimalna horizontalna koordinata, y_{min} gornji rub ili minimalna vertikalna koordinata, x_{max} desni rub ili

maksimalna horizontalna koordinata, y_{max} donji rub ili maksimalna vertikalna koordinata. Još jedan način zapisa je isti kao prvi, samo što se x i y odnose na centar okvira, a w i h na ukupnu širinu i visinu graničnog okvira.

Osim zapisa mjesta, potrebno je dodati zapis za postojanje objekta na slici, koji se označava s p_c te može poprimiti broj 0, u slučaju kada ne postoji objekt od interesa, ili broj 1, kada postoji. U slučaju jednostavnog primjera na slici 2.2., zapis bi bio 1.

Is there a **car** in this image?



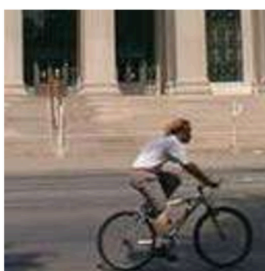
→ Yes

slika 2.2. postojanje objekta od interesa

Naravno, u slučaju s više različitih objekata, potrebno je zapisati koji je objekt pronađen. Ovisno o broju različitih objekata, taj zapis se izražava s $C = \{c_1, c_2, \dots, c_k\}$ gdje k predstavlja broj različitih objekata, a mogu poprimiti vrijednost 0 ili 1 u slučaju kada specifičan objekt ne postoji, odnosno, postoji.



$\begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}$



$\begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}$



$\begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}$



$\begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}$



$\begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$



$\begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$

slika 2.3. zapis klasifikacije različitih objekata od interesa

U primjerima na slici 2.3., kada postoje 3 klase, koje se kodiraju one-hot encoding načinom, tada bi k bio jednak 3, a zapis bi se mijenjao s obzirom koja je klasa pronađena.

Lokacija se ovisno o načinu zapisa, izražava s $B = (b_x, b_y, b_h, b_w)$. Napokon, kada se zapisi svedu na jedan, tada za svaku sliku postoji $y = [p_c, b_x, b_y, b_h, b_w, c_1, c_2, \dots, c_k]$.

Funkcija gubitka odnosno mjera odstupanja između točne vrijednosti i vrijednosti koje je model zapisao zapisuje se:

$$L(\hat{y}, y) = L(\hat{y}_1, y_1) + L(\hat{y}_{2-5}, y_{2-5}) + L(\hat{y}_{6-k}, y_{6-k}) \quad (2-1)$$

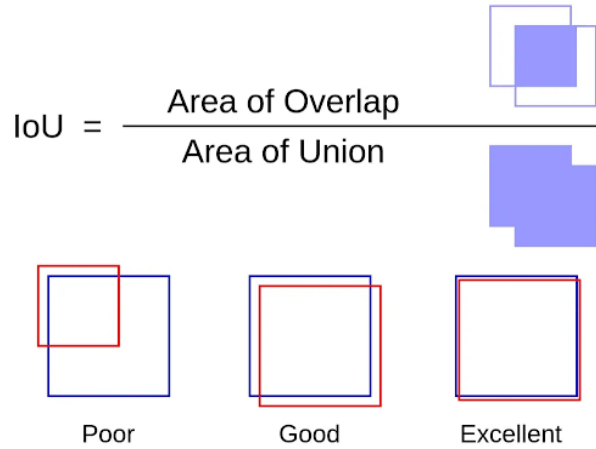
gdje su oznake s $\hat{y}$ predviđene od strane modela, a y isnite vrijednosti. Može se primijetiti kako postoje tri funkcije funkcije gubitka, gdje je prva funkcija nakon izraza jednakosti vjerojatnost pojavljivanja objekta, druga funkcija gubitka se odnosi na poziciju graničnog okvira te posljednja je funkcija gubitka za objekte. Također, funkcije gubitka se razlikuju po tome što se prema poziciji graničnog okvira gleda kao regresijski problem, dok se na prepoznavanje objekata gleda kao klasifikacijski problem te se trebaju primijeniti gubitci koji odgovaraju takvim problemima, kao L2 gubitak za regresiju, a gubitak unakrsne entropije za klasifikaciju.

Za lokalizaciju, često se koristi metrika presjek preko unije (engl. Intersection Over Union - IoU) ili Jaccardov index. Metrika koristi omjer presjeka predviđenog i stvarnog graničnog okvira te uniju predviđenog i stvarnog graničnog okvira, matematički zapisanog:

$$IoU = \frac{A \cap B}{A \cup B} \quad (2-2)$$

gdje je A stvarni granični okvir, a B predviđeni granični okvir. Pomoću ove metrike može se dobiti broj u intervalu 0 i 1 koji se interpretira koliko dobro predviđeni okvir odgovara stvarnom okviru, gdje 1 odgovara savršenom preklapanju, a 0 odgovara okvirima koji nemaju zajedničku površinu.

Primjeri vizualizacije ove metrike može se vidjeti na slici 2.4.. Može se primijetiti kako se s obzirom na razinu preklapanja može se reći koliko dobro se preklapaju granični okviri.



slika 2.4. vizualizacija interpretacije metrike presjeka preko unije

Primjenom IoU, može se dobiti osnovna funkcija gubitka:

$$L(\hat{y}_{2-5}, y_{2-5}) = 1 - IoU(\hat{y}_{2-5}, y_{2-5}) \quad (2-3)$$

gdje je, kao i u formuli 2-1, $\hat{y}$ oznaka za predviđeni granični okvir, a y oznaka za stvarni granični okvir. Funkcija gubitka želi se smanjiti, što znači kada se okviri preklapaju, tj. kada je IoU jednak 1, tada je funkcija gubitka 0. Naravno, funkcija gubitka se može modificirati kako bi se dobio bolji rezultat ta specifičan problem.

Funkcije gubitka kod klasifikacijskog dijela problema može se izraziti osnovnom binarnom unakrsnom entropijom, koja za primjer prve funkcije gubitka u 2-1 funkciji, glasi:

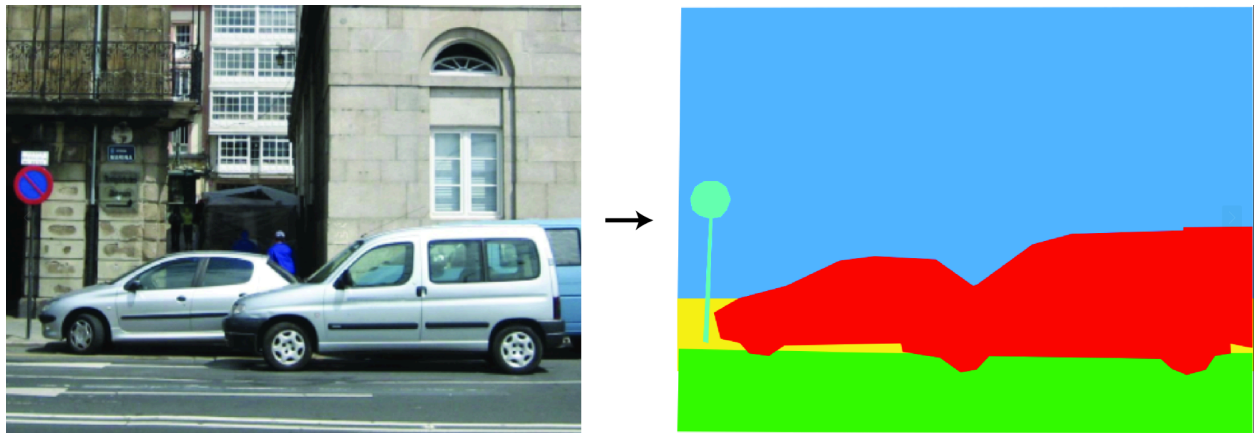
$$L(\hat{y}_1, y_1) = - (y_1 \log(\hat{y}_1) + (1 - y_1) \log(1 - \hat{y}_1)) \quad (2-4)$$

gdje se kao i prijašnjim funkcija, oznake $\hat{y}_1$ označavaju predviđene vrijednosti vjerojatnosti postojanja objekta, a y_1 oznake stvarne vrijednosti postojanja objekta. Za primjer s više klasa tj. na primjeru treće funkcije u formuli 2-1, može se koristiti osnovna unakrsna entropija koja glasi:

$$L(\hat{y}_c, y_c) = - \sum_{c=1}^k y_c \log(\hat{y}_c) \quad (2-5)$$

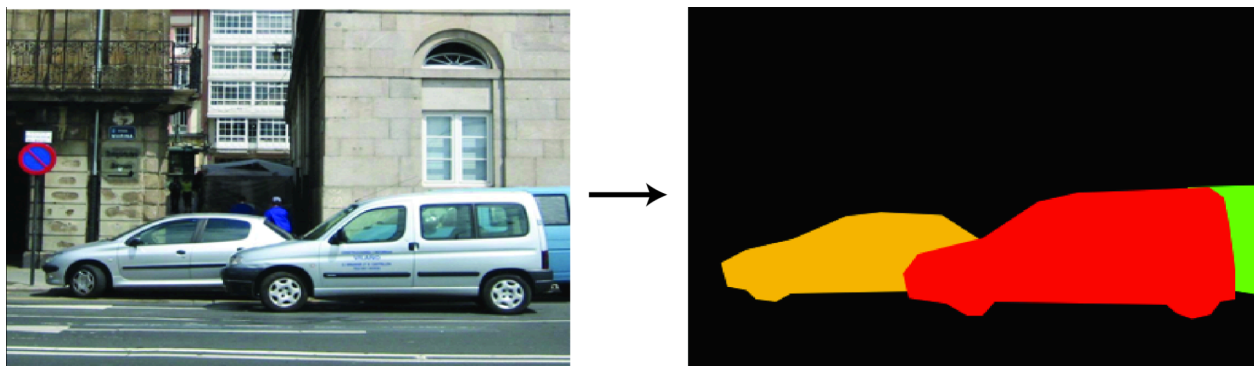
gdje je $\hat{y}_c$ predviđena vrijednost vjerojatnosti određene klase, y_c stvarna vrijednost određene klase, k je ukupan broj klasa, a c skup različitih objekata.

Nakon rješavanja problema klasifikacije i lokalizacije, može se ići korak dalje, na semantičku segmentaciju. Semantička segmentacija dodjeljuje zajedničke oznake ili kategorije svakom pikselu na slici, što znači kako je ovaj korak precizniji od korištenja graničnih okvira, uz to jasnije otkriva grupe od interesa koje se nalaze na slici. Pikseli koji pripadaju istoj grupi od interesa, klasi, označavaju se istom bojom.



slika 2.5. primjer lokalizacije objekta od interesa

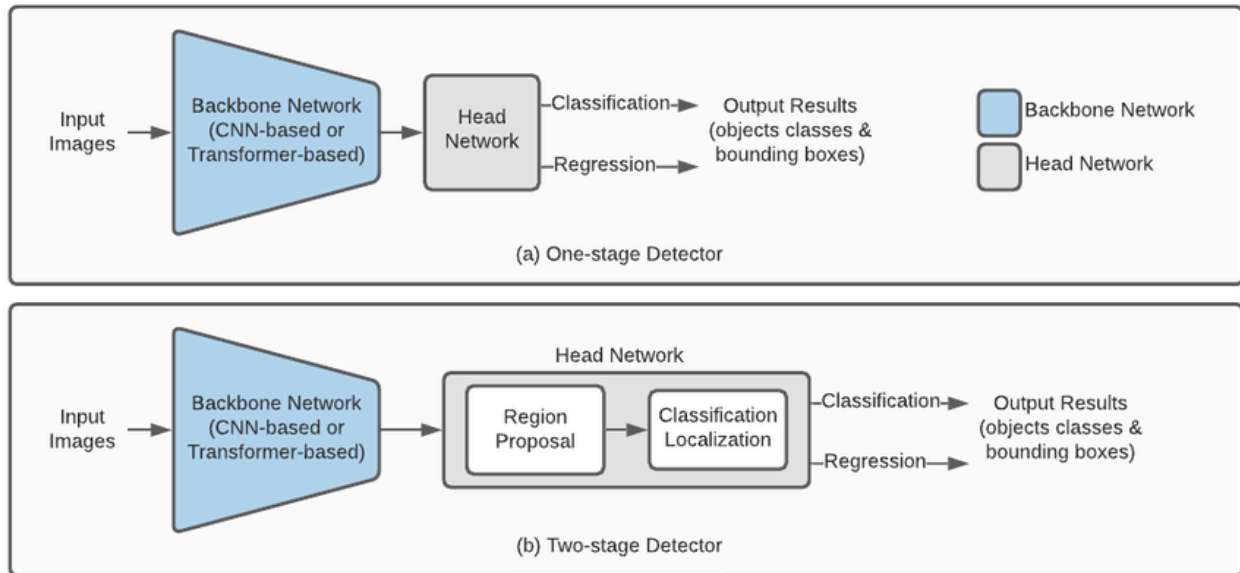
Idući korak semantičke segmentacije je segmentacija na instance. Uz sve pozitivne strane semantičke segmentacije, razlikuje se različito instancirani objekti iste semantičke klase, označenim drugačijom bojom piksela u prikazima.



slika 2.6. primjer lokalizacije objekta od interesa

2.2. Moderni detektori objekata

Nakon definiranja detekcije objekata i segmentacije, moderni detektori mogu se podijeliti na dvije skupine, one stage i two stage detektori.



slika 2.7. vizualizacija osnovne arhitekture one stage i two stage detektora

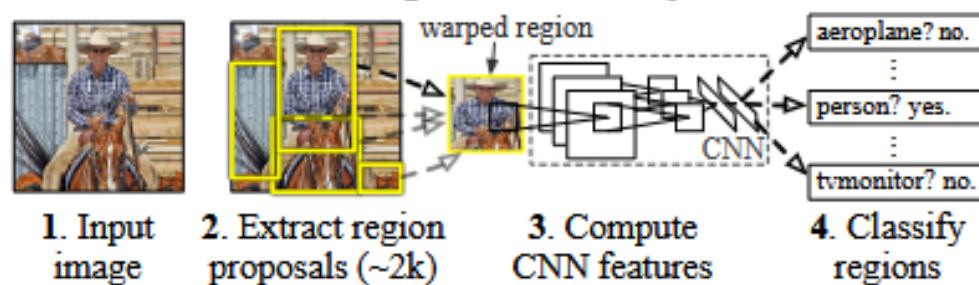
Two stage detektori imaju stadij gdje se prijedlažu regije koje bi mogle sadržavati objekte. Jedna vizualizacija primjera algoritma za predlaganje prijedloga regije sadržavanja objekta je prikazana na slici 2.8.



slika 2.8. vizualizacija primjera algoritma prijedloga regije

Ovakav algoritam prvo pretjerano segmentira sliku, a rijekom obrade regija dolazi do većih regija u kojima se mogu nalaziti objekti. Grupiraju se s obzirom na 4 mjere, sličnost boje, teksture, oblika i veličine. Ovakav pristup omogućuje detektorima usredotočenje na manji broj regija, što ubrzava proces detekcije. Takve regije se mogu shvatiti kao isječci slike koji, iduće, prolaze kroz mrežu za klasifikaciju i lokalizaciju.

Primjer two stage detektora je R-CNN, prikazan na slici 2.9., koji predlaže 2000 regija po slici, a konvolucijskom mrežom izdvaja značajke koje ulaze u SVM (engl. Support Vector Machines) kako bi se klasificirala regija i u linearni regresijski model za procjenu graničnog okvira.



slika 2.9. vizualizacija rada R-CNN detektora

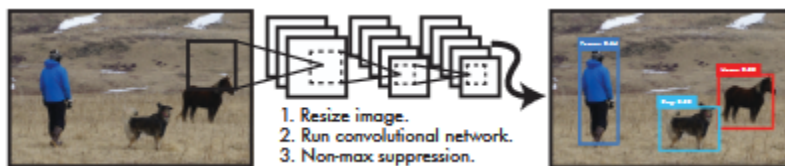
Prolazak kroz regije pa detekcija objekta omogućuje ovakvim detektorima visoku preciznost lokalizacija i manji broj lažno pozitivnih klasifikacija, ali upravo taj prolazak uzima puno vremena što znači kako ne mogu biti korišteni u stvarnom vremenu. Uz vrijeme izvedbe, njihova arhitektura je složenija, zahtijeva puno memorije za skladištenje podataka i treniranje je sporo.

Osim R-CNN, koji je objavljen 2014. godine, postoje još neki primjeri two stage detektora poput: SPP-Net (2014.), Fast R-CNN (2015.), Faster R-CNN (2015.), FPN (2016.), DetectorRS (2020.). Primjeri one stage detektora su SSD (2015.), RetinaNet (2017.), CenterNet (2019.), a najpoznatiji one stage detektor je YOLO koji je originalno predstavljen 2015. godine.

One stage detektori, za razliku od two stage, nemaju stadij prijedloga regije gdje bi objekt mogao biti sadržan, već izravno predviđaju granične okvire i klasu objekata u jednom prolazu kroz mrežu. Takav pristup omogućuje detekciju u jednom koraku što rezultira većim brzinama inferencije i jednostavnijem treniranju. Rast u brzini omogućuje i rad u stvarnom vremenu što ih čini pogodnim za mnoge probleme. Iako su brži, ta brzina povećava mogućnost greške, pogotovo kod objekata koji se preklapaju ili su mali.

2.2.1. YOLOv1

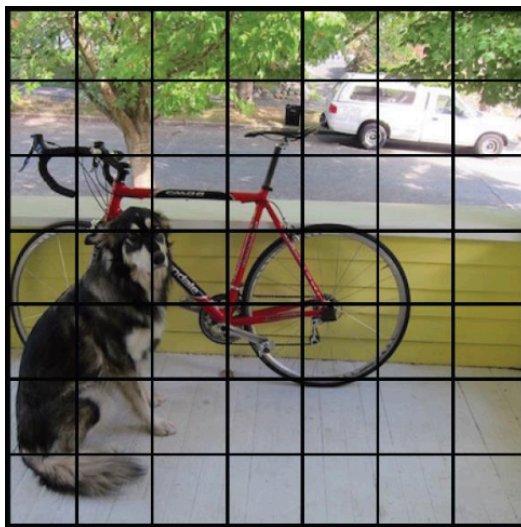
Prva verzija YOLO modela mijenja veličinu ulazne slike na 448x448 piksela, predaju ju konvolucijskoj mreži te rezultirane detekcije odbacuje s obzirom na prag modelove sigurnosti. Vizualizacija takvog rada modela može se pronaći na slici 2.10.



slika 2.10. vizualizacija rada YOLOv1 detektora

Takav način rada omogućuje iznimno brzu obradu slika, 45 slika po sekundi, dok za verziju manje mreže, Fast YOLO, obrada slike se događa još brže, 155 slika po sekundi, uz takvu brzinu, Fast YOLO postiže dvostruko veći mAP (engl. mean Average Precision) od drugih modela njegovog vremena.

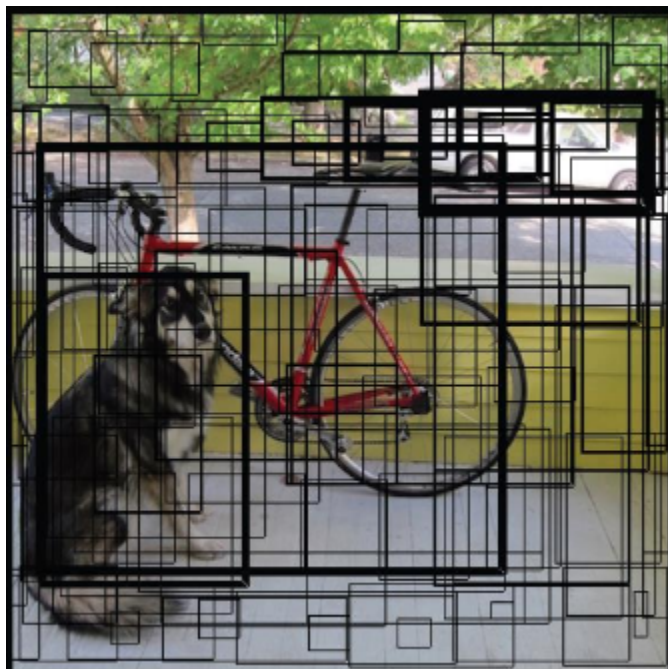
Mreža koristi značajke cijele slike kako bi predviđela svaki granični okvir. Također predviđa sve granične okvire svake klase istovremeno. Model dijeli sliku na $S \times S$ rešetku, a ako centar objekta leži u ćeliji rešetke, ta ćelija je odgovorna za detekciju tog objekta. Takva podjela se može vidjeti na slici 2.11.



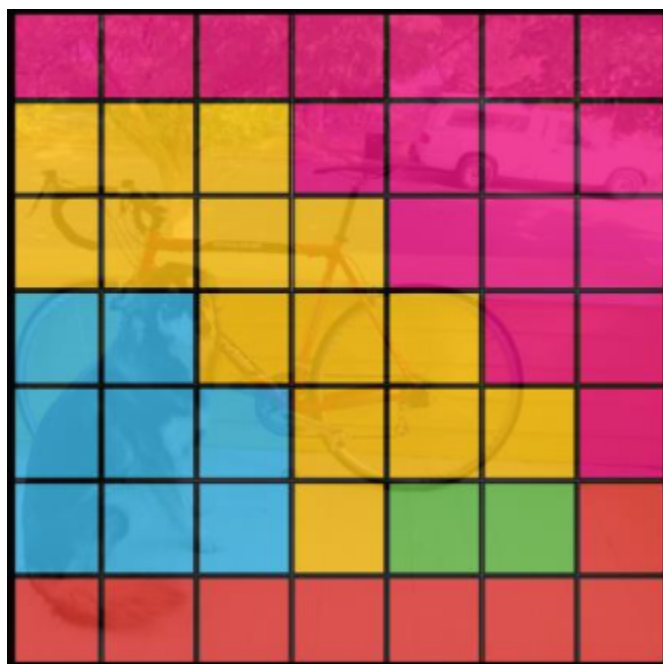
slika 2.11. vizualizacija podjele slike na rešetke

Svaka rešetka predviđa B graničnih okvira i sigurnost odluke za te okvire. Sigurnost odluke označava koliko je model siguran postoji li objekt u tom okviru i koliko je precizan takav okvir. U ovom slučaju granični okvir se sastoji od 5 predikcija, x , y , w , h i *confidence*. Koordinate (x,y) predstavljaju centar okvira u odnosu s granicama ćelije rešetke. Oznake w i h predstavljaju

širinu i visinu u odnosu sa cijelom slikom, a *confidence* predstavlja IOU između predviđenog okvira i stvarnog okvira. Vizualizacija predviđenih okvira i sigurnosti odluke je prikazano na slici 2.12.



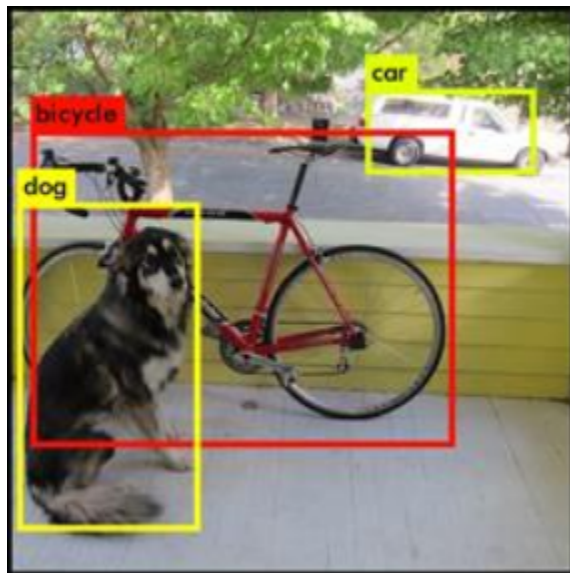
slika 2.12. vizualizacija predviđenih graničnih okvira i sigurnosti odluke po ćeliji



slika 2.13. vizualizacija vjerojatnosti klasa po ćeliji rešetke

Uz predviđanje graničnih okvira i sigurnosti odluke, svaka ćelija rešetke predviđa i vjerojatnost sadržavanja klasa C. Taj dodatak predstavlja sigurnost odluke za specifičnu klasu objekta za svaki okvir. Rezultirajuća vizualizacija vjerojatnosti klase objekta po ćeliji rešetke može biti pronađena na slici 2.13.

Krajnja vizualizacija inferencije modela koja nakon donošenja odluke primjenjuje potiskivanje nemaksimalnih vrijednosti kako bi se smanjio broj praplapajućih graničnih okvira, je prikazan na slici 2.14. Primjena potiskivanja nemaksimalnih vrijednosti daje 2-3% mAP-u.



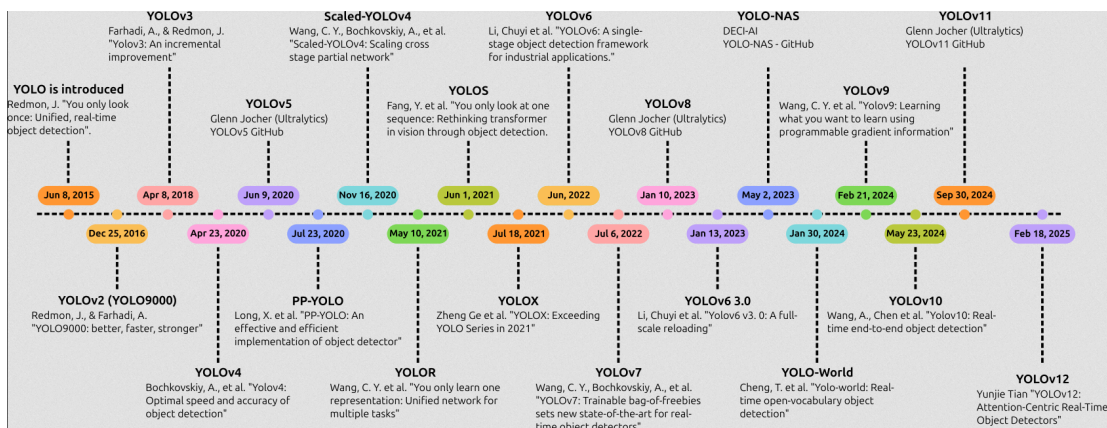
slika 2.14. vizualizacija inferencije modela

2.3. Razvoj arhitekture YOLO modela

Radi iznimne brzine koja je omogućila YOLOv1 modelu primjenu u stvarnom vremenu, koju drugi modeli u vremenu objave YOLOv1 modela nisu postizali, ali i dovoljno dobre preciznosti, nastavio se razvoj na YOLO modelu koja je tijekom godina nizala razne verzije za generalnu primjenu, ali i rješavanje specifičnih problema, a pregled tijeka razvoja modela je prikazan na slici 2.15.

2.3.1. YOLOv1

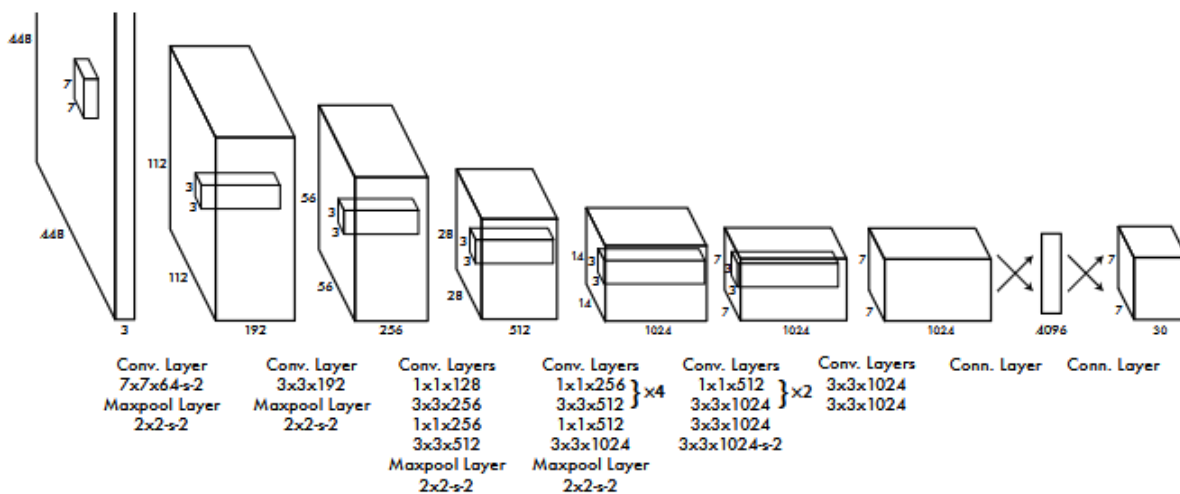
Arhitektura mreže YOLOv1 modela sastoji se od 24 sloja konvolucije koja završava s 2 potpuno povezana sloja. Koriste se 1x1 redukcijski slojevi praćeni 3x3 konvolucijskim slojevima. Usporedno, Fast YOLO koristi 9 konvolucijski slojeva i manje filtera u slojevima, ali ostali parametri su isti.



slika 2.15. tijek razvoja YOLO modela kroz godine

Na kraju svake predikcije, izlaz je zapisan kao $SxSx(B * 5 + C)$ tenzor. U originalnom radu koristi se $7x7x(2 * 5 + 20)$ tenzor. Također su konvolucijski slojevi bili prethodno trenirani na ImageNet klasifikacijskim zadacima na rezolucijama 224x224 ulaznih slika, čije se rezolucije nakon bile udvostručene radi detekcije. Vizualizacija arhitekture mreže je prikazana na slici 2.16.

Za treniranje se koristila funkcija gubitka koja se sastojala od regresijskog gubitka, klasifikacijskog gubitka te gubitka sigurnosti odluke.



slika 2.16. arhitektura YOLOv1

2.3.2. YOLOv2

YOLOv2 koristi novi model imenom Darknet-19, koji je prikazan na slici 2.17. On ima 19 slojeva konvolucije i 5 maxpool sloja. Uvodi batch normalizaciju za svaki konvolucijski sloj te miče potpuno povezane slojeve, što čini ovu mrežu potpuno konvolucijskom, što smanjuje broj parametara i ubrzava inferenciju.

Num	Type	Filters	Size/Stride	Output
1	Conv/BN	32	$3 \times 3 / 1$	$416 \times 416 \times 32$
2	Max Pool		$2 \times 2 / 2$	$208 \times 208 \times 32$
3	Conv/BN	64	$3 \times 3 / 1$	$208 \times 208 \times 64$
4	Max Pool		$2 \times 2 / 2$	$104 \times 104 \times 64$
5	Conv/BN	128	$3 \times 3 / 1$	$104 \times 104 \times 128$
6	Conv/BN	64	$1 \times 1 / 1$	$104 \times 104 \times 64$
7	Conv/BN	128	$3 \times 3 / 1$	$104 \times 104 \times 128$
8	Max Pool		$2 \times 2 / 2$	$52 \times 52 \times 128$
9	Conv/BN	256	$3 \times 3 / 1$	$52 \times 52 \times 256$
10	Conv/BN	128	$1 \times 1 / 1$	$52 \times 52 \times 128$
11	Conv/BN	256	$3 \times 3 / 1$	$52 \times 52 \times 256$
12	Max Pool		$2 \times 2 / 2$	$52 \times 52 \times 256$
13	Conv/BN	512	$3 \times 3 / 1$	$26 \times 26 \times 512$
14	Conv/BN	256	$1 \times 1 / 1$	$26 \times 26 \times 256$
15	Conv/BN	512	$3 \times 3 / 1$	$26 \times 26 \times 512$
16	Conv/BN	256	$1 \times 1 / 1$	$26 \times 26 \times 256$
17	Conv/BN	512	$3 \times 3 / 1$	$26 \times 26 \times 512$
18	Max Pool		$2 \times 2 / 2$	$13 \times 13 \times 512$
19	Conv/BN	1024	$3 \times 3 / 1$	$13 \times 13 \times 1024$
20	Conv/BN	512	$1 \times 1 / 1$	$13 \times 13 \times 512$
21	Conv/BN	1024	$3 \times 3 / 1$	$13 \times 13 \times 1024$
22	Conv/BN	512	$1 \times 1 / 1$	$13 \times 13 \times 512$
23	Conv/BN	1024	$3 \times 3 / 1$	$13 \times 13 \times 1024$
24	Conv/BN	1024	$3 \times 3 / 1$	$13 \times 13 \times 1024$
25	Conv/BN	1024	$3 \times 3 / 1$	$13 \times 13 \times 1024$
26	Reorg layer 17			$13 \times 13 \times 2048$
27	Concat 25 and 26			$13 \times 13 \times 3072$
28	Conv/BN	1024	$3 \times 3 / 1$	$13 \times 13 \times 1024$
29	Conv	125	$1 \times 1 / 1$	$13 \times 13 \times 125$

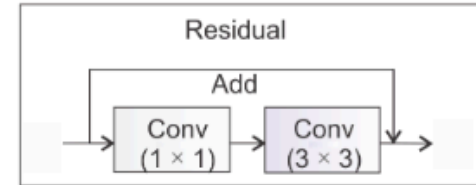
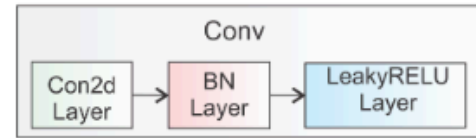
slika 2.17. YOLOv2 arhitektura

Za razliku od prvog modela, drugi model je unaprijed treniran na slikama 448x448 rezolucije, što poboljšava rezultat na ulazima više rezolucije. Koristi se *anchor box* koji je prijedlog graničnim okvirima, koji služi za ubrzanje inferencije mreže. Povećava se broj graničnih okvira po ćeliji na 5. Koristi se *passthrough* sloj koji organizira okolne značajke mapi značajki u druge kanale umjesto gubljenja značajki kroz prostorno poduzorkovanje.

2.3.3. YOLOv3

YOLOv3 model uvodi *objectness score* metriku, koja koristi logističku regresiju, za svaki granični okvir. Umjesto korištenja *softmax* za klasifikaciju, koristi se BCE (engl. binary cross-entropy) za treniranje neovisnog klasifikatora koji predstavlja problem kao klasifikacija s više oznaka, što omogućuje dodijeljivanje više oznaka za isti granični okvir. Koristi se Darknet-53 kao temelj arhitekture YOLOv3 modela, koji se sastoji od 53 konvolucijska sloja sa residualnim vezama, prikaz Darknet-53 se može naći na slici 2.18.

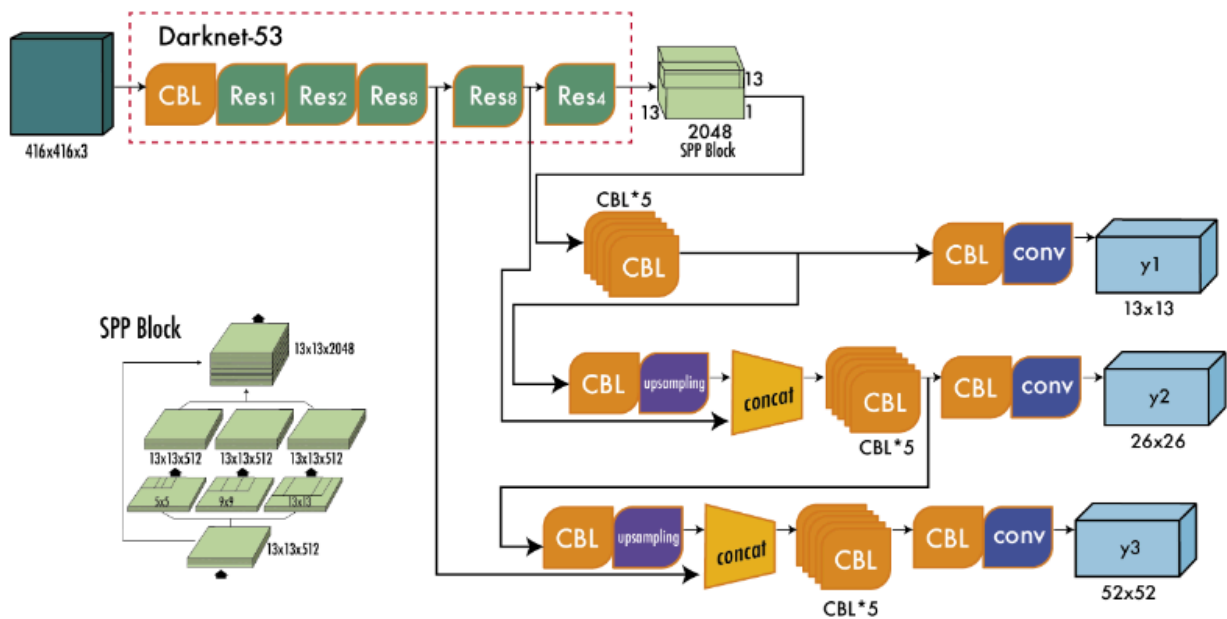
Layer	Filters size	Repeat	Output size
Image			416×416
Conv	32 $3 \times 3/1$	1	416×416
Conv	64 $3 \times 3/2$	1	208×208
Conv	32 $1 \times 1/1$	Conv Conv Residual	208×208
Conv	64 $3 \times 3/1$		208×208
Residual			208×208
Conv	128 $3 \times 3/2$	1	104×104
Conv	64 $1 \times 1/1$	Conv Conv Residual	104×104
Conv	128 $3 \times 3/1$		104×104
Residual			104×104
Conv	256 $3 \times 3/2$	1	52×52
Conv	128 $1 \times 1/1$	Conv Conv Residual	52×52
Conv	256 $3 \times 3/1$		52×52
Residual			52×52
Conv	512 $3 \times 3/2$	1	26×26
Conv	256 $1 \times 1/1$	Conv Conv Residual	26×26
Conv	512 $3 \times 3/1$		26×26
Residual			26×26
Conv	1024 $3 \times 3/2$	1	13×13
Conv	512 $1 \times 1/1$	Conv Conv Residual	13×13
Conv	1024 $3 \times 3/1$		13×13
Residual			13×13



slika 2.18. Darknet-53

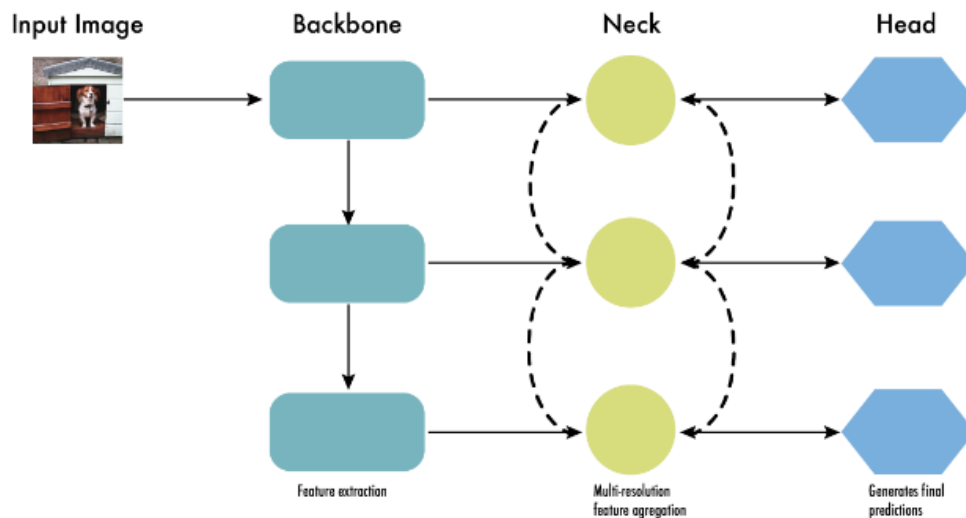
Uz drugačiji temelj, dodaje se SPP (engl. *Spatial Pyramid Pooling*) koji nadovezuje više *max pool* sloja bez poduzorkovanja, sa različitim veličinama jezgre $k \times k$ gdje je $k = 1, 5, 9, 13$, što rezultira većim receptivnim poljem. Takva verzija modela naziva se YOLOv3-spp, što je najbolji YOLOv3 model. Slično FPN (engl. *Feature Pyramid Network*), YOLOv3 predviđa 3 granična okvira u 3 različite skale. To pomaže modelu u predikciji malih objekata, što je bio problem ranijih verzija.

Prema slici 2.19., koja prikazuje arhitekturu YOLOv3 modela, y1 izlaz je jednak izlazu YOLOv2 modela, definiran 13×13 rešetki ćelija, y2 izlaz sastoji se od nadovezivanja izlaza (Resx4) s (Resx8) Darknet-53 temelja, gdje mapa značajki mora imati drugačiju veličinu kako bi se povećala prije nadovezivanja. Napokon, y3 izlaz nadovezuje 26×26 i 52×52 mapu značajki.



slika 2.19. arhitektura YOLOv3

Prema slici 2.20., može se vidjeti kako se u modernim detektorima objekata koristi sažeti dizajn arhitekture gdje se koristi temelj poput konvolucijskih neuronskih mreža, kako bi se izvukle značajke slike u različitim skalama, vrat koji poboljšava te značajke prostornim i semantičkim informacijama te glava koja dodatno poboljšava značajke kako bi se mogla dati pretpostavka detektiranog objekta.



slika 2.20. arhitektura modernih detektora objekata

2.3.4. YOLOv4

YOLOv4 je pokušao pronaći ravnotežu oko mnogih eksperimenata kroz *bag-of-freebies*, metode koje mijenjaju strategiju treniranja i povećava cijenu treniranja, ali ne i njenu inferenciju, i *bag-of-specials*, metode koje povećavaju vrijeme inferencije, ali uvelike povećavaju preciznost.

Nakon mnogo testiranja, pronađen je najbolji temelj, Darknet-53 sa CPSNet (engl. *Cross-Stage Partial Connections*) i Mish aktivacijskom funkcijom. Za vrat se koristi SPP od YOLOv3-spp i predikcije s više skala od YOLOv3, ali s izmijenjenom inačicom PANet (engl. *Path Aggregation Network*) gdje se umjesto FPN koristi izmijenjeni SAM (engl. *Spatial Attention Module*). Napokon, za vrat se koriste *anchor box* kao i u YOLOv3. Model je stoga nazvan CSPDarknet53-PANet-SPP.

CSP smanjuje izračun modela bez smanjenja preciznosti, SPP povećava receptivno polje, a PANet nadovezuje značajke.

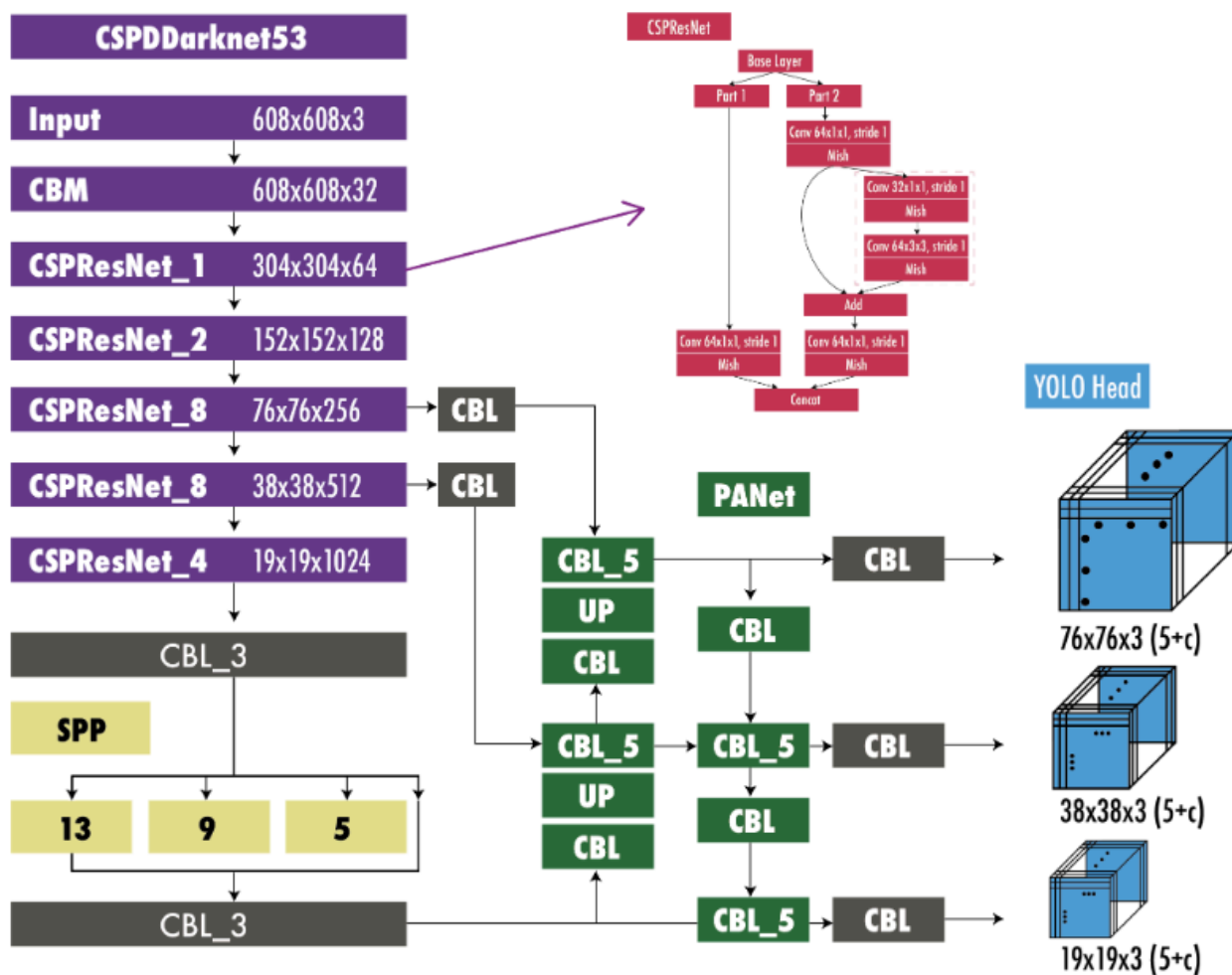
Backbone	Detector
Bag-of-Freebies	Bag-of-Freebies
Data augmentation	Data augmentation
- Mosaic	- Mosaic
- CutMix	- Self-Adversarial Training
Regularization	CIoU loss
- DropBlock	Cross mini-Batch Normalization (CmBN)
Class label smoothing	Eliminate grid sensitivity
	Multiple anchors for a single ground truth
	Cosine annealing scheduler
	Optimal hyper-parameters
	Random training shapes
Bag-of-Specials	Bag-of-Specials
Mish activation	Mish activation
Cross-stage partial connections	Spatial pyramid pooling block
Multi-input weighted residual connections	Spatial attention module (SAM)
	Path aggregation network (PAN)
	Distance-IoU Non-Maximum Suppression

slika 2.21. konačan odabir *bag-of-freebies* i *bag-of-specials*

Slika 2.21. prikazuje odabir metoda poboljšanja koji se sastoje od osim standardnih augmentacija ulaza, koristi se mozaična augmentacija gdje se 4 slike spajaju u jednu, radi smanjivanja potrebe za velikim *mini-batchom* kod *batch* normalizacije. Za regularizaciju koristi se *DropBlock*, kao zamjena za *Dropout*, a za detektore se koristi CIoU (engl. Complete Intersection Over Union) gubitak i CmBN (engl. Cross mini-batch Normalization).

Koristi se i SAT (engl. Self-adversarial Training) kako bi model bio otporniji na poremećaje i koristi optimizaciju hiperparametara pomoću genetskog algoritma.

Na slici 2.22., prikazana je arhitektura YOLOv4 mreže gdje CMB spaja konvoluciju, batch normalizaciju i Mish aktivacijsku funkciju, CBL spaja konvoluciju, batch normalizaciju i Leaky ReLU (engl. Rectified Linear Unit) aktivacijsku funkciju, UP je povećanje uzorkovanja, a ostale oznake su otprije poznate.

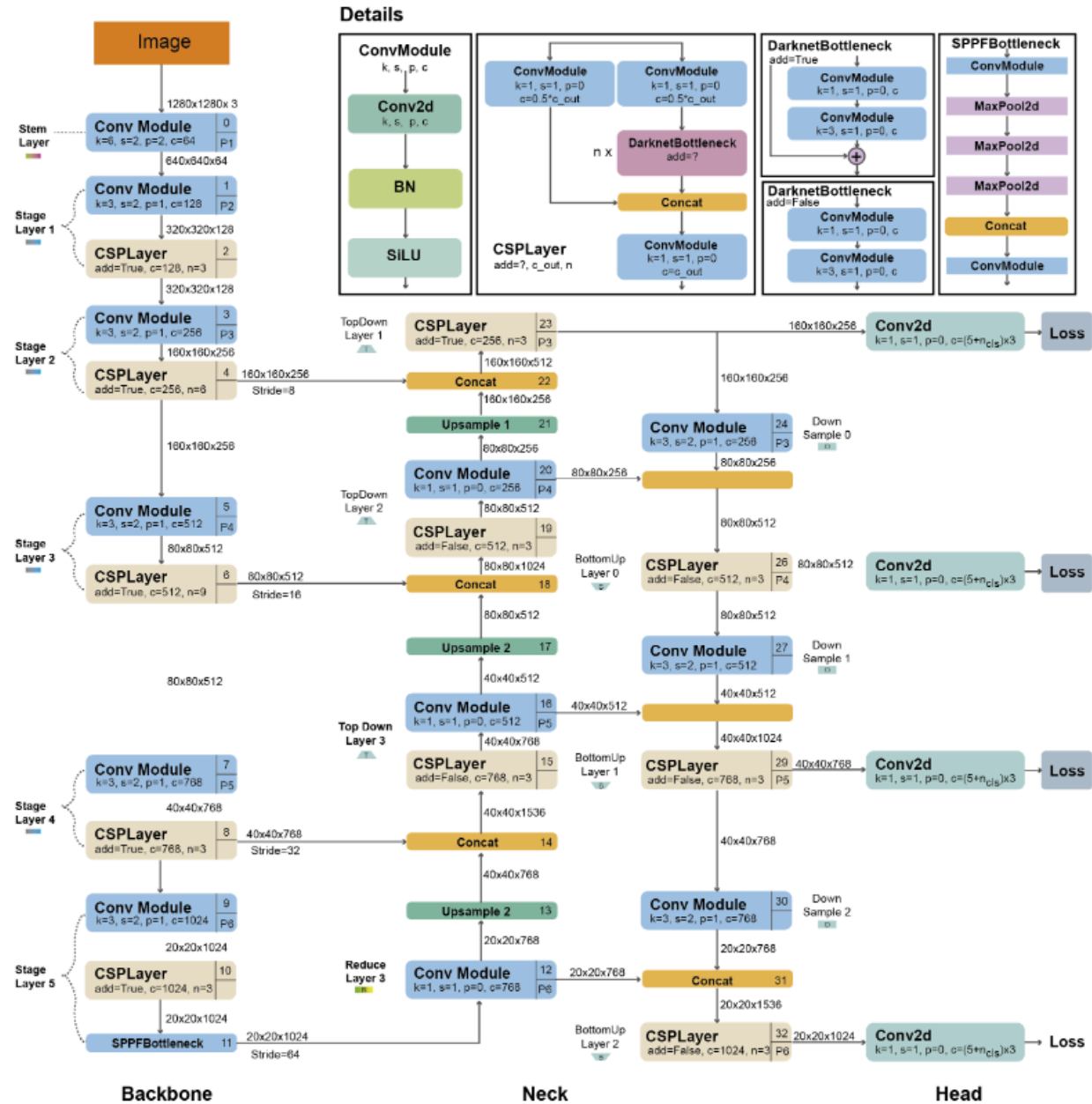


slika 2.22. arhitektura YOLOv4

2.3.5. YOLOv5

Posebnost ovog modela nalazi se u algoritmu *Ultralytics*, AutoAnchor, koji, prije treniranja, provjerava jesu li *anchor box*evi primjereni skupu podataka i postavkama treninga. Koristi augmentacije poput mozaika, *copy paste*, *random affine*, MixUp, HSV (engl. Hue, Saturation, Value) augmentacija, nasumični horizontalni preokret i ostale augmentacije paketa *albumentations*.

Arhitektura modela koristi izmijenjenu CSPDarknet53 koja počinje sa Stem slojem, *strided* konvolucijski slojem s velikim iznosom prozora kako bi se smanjila cijela memorije i izračuna. Slijede konvolucijski slojevi za vađenje značajki ulazne slike. SPPF (engl. *Spatial Pyramid Pooling Fast*) sloj i idući konvolucijski slojevi obrađuju značajke dok slojevi za povećanje uzorkovanja povećavaju rezoluciju mape značajki. SPPF ubrzava izračun skupljanjem značajki u mapu značajki fiksne veličine. Svaka konvolucija je praćena BN (engl. *Batch Normalization*) i SiLU (engl. *Sigmoid Linear Unit*) aktivacijskom funkcijom.

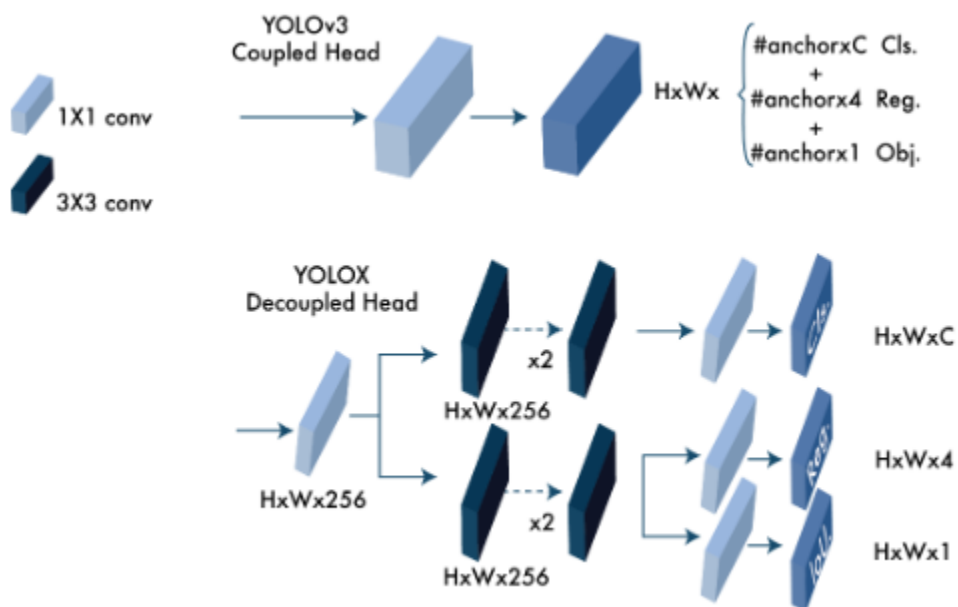


slika 2.23. arhitektura YOLOv5

Vrat koristi SPPF i izmijenjeni CSP-PAN (engl. *Cross Stage Partial Path Aggregation Network*), dok je glava slična YOLOv3. Cijela arhitektura YOLOv5 je prikazana na slici 2.23.

2.3.6. YOLOv6

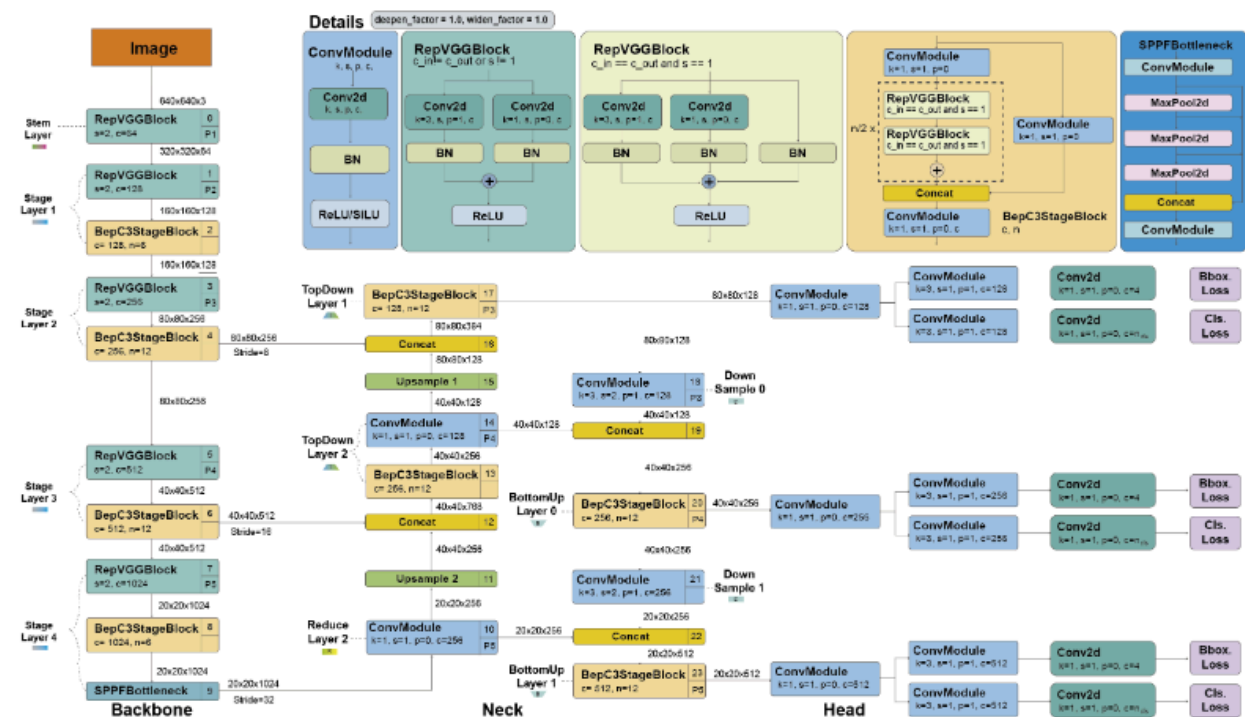
Temelj šeste verzije YOLO modela miče se od Darkneta i prelazi se na novi temelj baziran na RepVGG, *EfficientRep*, koji koristi veću količinu paralelizma od prijašnjih temelja modela. Za vrat koristi se PAN poboljšan *RepBlockovima* ili *CSPStackRep*. Glava je razdvojena, kao što je prikazano na slici 2.24.



slika 2.24. razlika spojene i razdvojene glave

Za dodijelu oznaka koristi se pristup *Task alignment learning* koji je predstavljen u TOOD (engl. *Task-aligned One-stage Object Detection*). Koriste se drugačije funkcije gubitka, VariFocal funkcija gubitka i SIOU (engl. *Scylla-Intersection Over Union*)/GIOU (engl. *Generalized-Intersection Over Union*) regresija. Za zadatke regresijskog i klasifikacijskog tipa koristi se *self-distilled* strategija, a za detekciju se koristi RepOptimizer i *channel-wise distillation*.

Kao što je prikazano na slici 2.25., osim razdvojene glave i korištenja novog temelja, RepVGG, SPPF i Conv moduli su slični kao u YOLOv5 modelu.



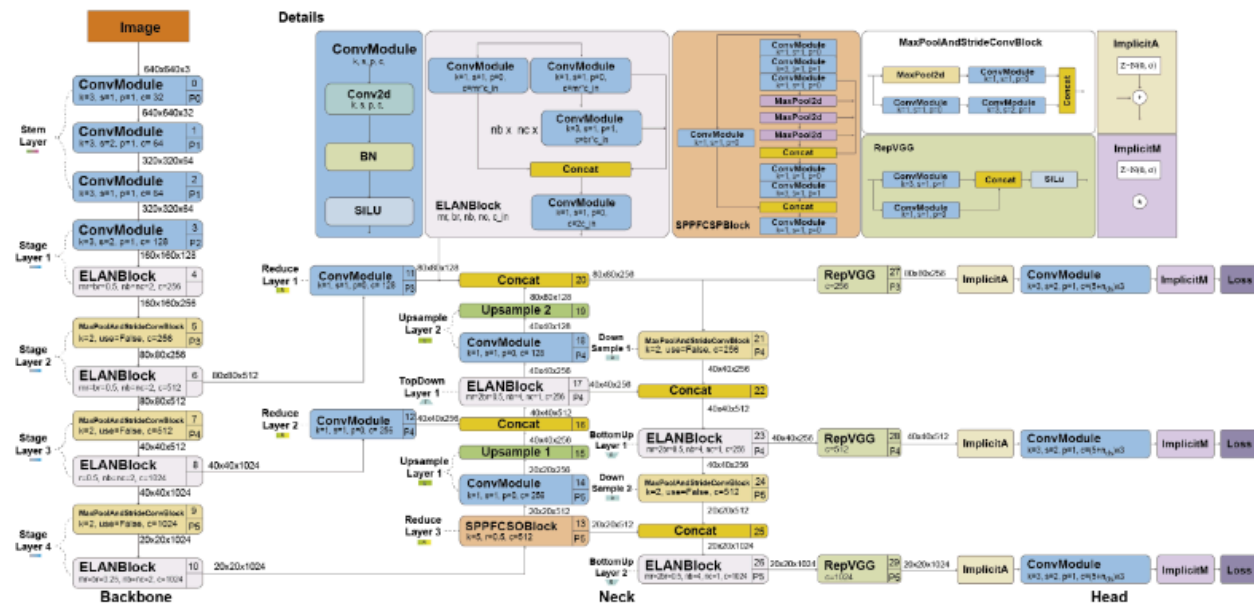
slika 2.25. arhitektura YOLOv6

2.3.7. YOLOv7

Novost koju donosi model je E-ELAN (engl. *Extended Efficient Layer Aggregation Network*). To je strategija koja omogućuje modelu brže učenje i konvergenciju upravljanjem nad putem gradijenta. Također, omogućuje skaliranje modela za modele bazirane na nadovezivanju. Takvo skaliranje omogućuje stvaranje modela različitih veličina prilagođavanjem svojstva modela. Standardne tehnike skaliranja, poput one po dubini, rezultiraju promjenom omjera između ulaznog i izlaznog kanala prijelaznog sloja, što dovodi do smanjenja mogućnosti korištenja resursa modela. Nova strategija predlaže skaliranje dubine i širine bloka s istim faktorom kako bi zadržala optimalna struktura modela.

Korišteni *bag-of-freebies* u modelu je RepConvN (engl. *Re-parameterized Convolution without identity*) koji je inspiriran RepConv (engl. *Re-parameterized Convolution*), ali identitetska veza uništava residual u ResNet (engl. *Residual Network*) i nadovezivanje u DenseNet (engl. *Densely Connected Convolutional Network*). Iduća metoda je promjena odluke glava, gdje glavna glava donosi odluku o krajnjem izlazu, a pomoćna pomaže s treningom. Još jedan dodatak je integracija prosjeka i varijance *batch* normalizacije u pomak i težine konvolucijskog sloja u stadiju inferencije.

Promjene u YOLOv7 arhitekturi su prikazane na slici 2.26., a one uključuju ELAN blokove koji spajaju značajke različitih grupa na način gdje miješaju i spajaju kardinalnost kako bi poboljšali učenje modela i izmijenili RepVGG bez povezivanja identiteta.



slika 2.26. arhitektura YOLOv7

2.3.8. YOLOv8

YOLOv8 koristi slični temelj kao i YOLOv5, ali dolazi s promjenama na CSPLayeru, koji se zove C2f (engl. *cross-stage partial bottleneck with two convolutions*) modul. On spaja značajke visoke razine sa kontekstualnim informacijama kako bi se poboljšala točnost detekcije.

Model koristi *anchor-free* model sa razdvojenim glavama koji neovisno obrađuju zadatke klasifikacije, regresije i svojstva objekta, kako bi se poboljšala točnost. Koristi se sigmoidna funkcija u izlaznom sloju za vjerojatnost sadržavanje objekta u graničnom okviru, a softmax funkcija za predviđanje vjerojatnosti klase.

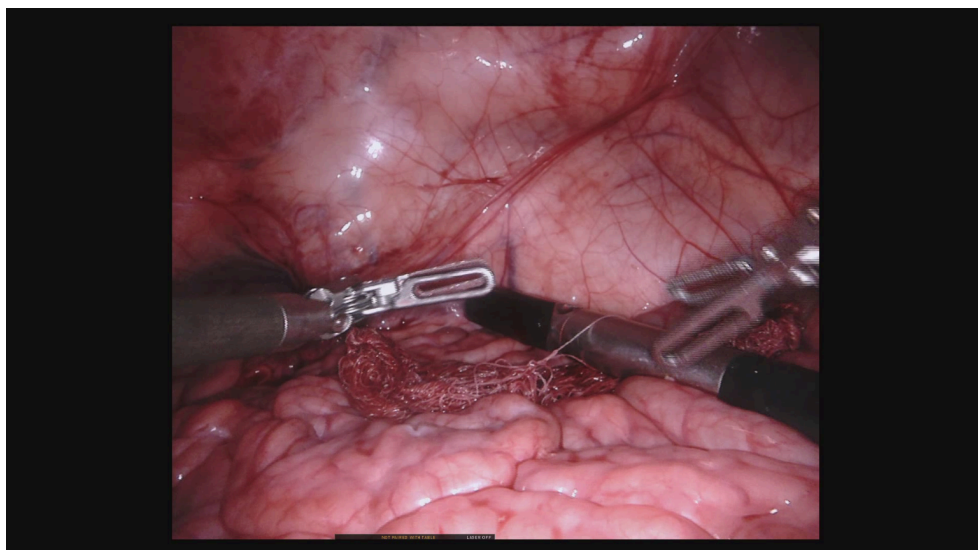
Koriste se Clou i DFL (engl. *Distribution Focal Loss*) funkcije gubitka za određivanje točnosti graničnih okvira, a BCE za klasifikaciju. Korištenje ovih funkcija gubitaka poboljšava rezultat kada se žele pronaći manji objekti.

Uz standardni model, ova inačica dolazi i s 5 skaliranim modelima, ali i modelom semantičke segmentacije, YOLOv8-Seg. Građen na temelju CSPDarknet53 i C2f modulom, praćenim s 2 segmentacijske glave kako bi se održala velika brzina i efikasnost.

Cijela arhitektura modela je prikazana na slici 2.27.

3. SKUP PODATAKA

Skup podataka namijenjen treniranju, sastoji se od 8 videozapisa po 225 slika. Slike su slikane da Vinci Xi robotom tijekom nekoliko zahvata na svinji. Videozapis je originalno bio snimljen na 30 Hz, ali je radi redundancije objavljen videozapis od 2 Hz. Primjer slike može biti viđen na slici 3.1.



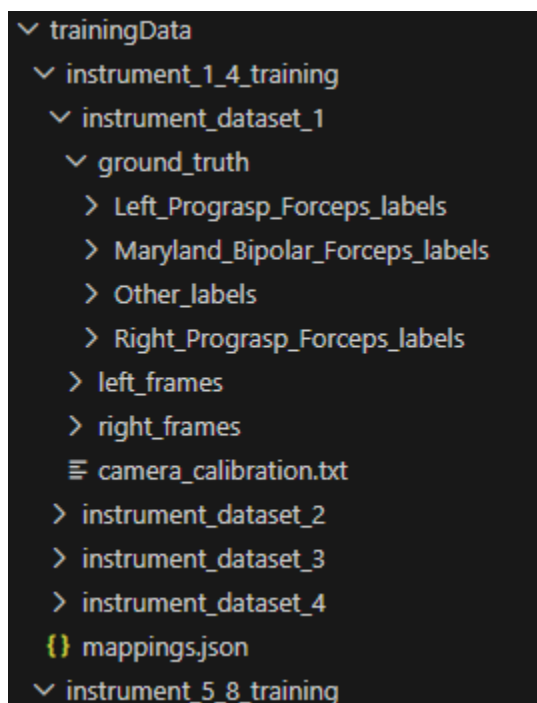
slika 3.1. primjer originalne slike skupa podataka za treniranje

Svaka slika u skupu za treniranje ima ručno označene dijelove robotskog kirurškog sustava, držalo (engl. *shaft*), zglob (engl. *wrist*), kopče (engl. *clasper*), kao na slici 3.2.



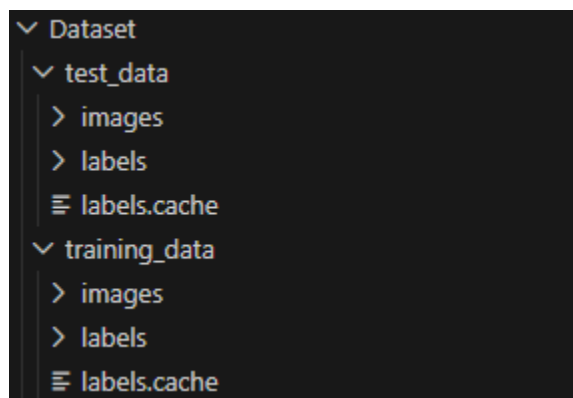
slika 3.2. primjer originalne slike oznake skupa podataka za treniranje

Također su označene dodatne kategorije kao instrument za laparoskopiju ili ultrazvučnu sondu. Skup podataka namijenjen testiranju, sastoji se od 8 videozapisa po 75 slika i 2 videozapisa po 300 slika. Oba videozapisa imaju jednaku frekvenciju, kao i videozapisi za treniranje.



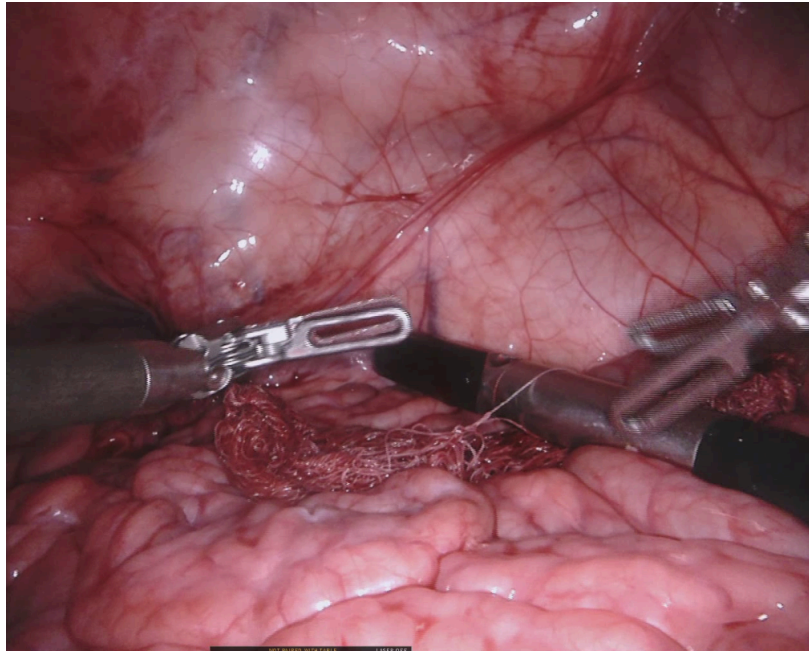
slika 3.3. originalna organizacija direktorija

Budući kako su slike već pripremljene iz videozapisa, potrebno je ih ispravno organizirati u direktorij kako bi YOLOv8 ispravno započeo učenje. To je napravljeno na način gdje direktorij *Dataset* sadrži 2 poddirektorij, *training_data* i *test_data*, a svaki ima direktorije, *labels* i *images* koji sadržavaju slike i oznake istih. Iz organizacije, kao na slici 3.3., napravljena je organizacija direktorija kao na slici 3.4.



slika 3.4. organizacija direktorija za YOLOv8

Budući kako je da Vinci robot opremljen stereo kamerom, u skupu podataka su uključene slike s lijeve strane, ali i desne. Originalne oznake su označene na slikama s lijeve strane, iako se mogu spojiti slike u jednu, radi izbjegavanja unošenja dodatnih grešaka, koristit će se slike uslikane s lijeve strane. Slike su rezolucije 1280x1024, ali pravi sadržaj slike počinje od piksela (320, 28). Iako oba skupa imaju takav raspored slika, nije nužno rezati sliku, ali radi uključivanja striktno nužnog sadržaja, predobrada se nastavlja s rezanjem crnih pravokutnika na slikama kako bi rezolucija bila 640x968.



slika 3.5. slika nakon micanja crnih pravokutnika

Kao iz prijašnjeg primjera označene slike, može se vidjeti kako nedostaju granični okviri. S obzirom na željeni rezultat i model koji se koristi potrebno je napraviti granične okvire iz slika kao što je slika 3.2. Za potrebe segmentacije i korištenja YOLOv8-se, nužno je koristiti okvire koji su poligoni koji prate obrub objekta.

Koristi se skripta koja ima predefinirane klase instrumenata poput *Bipolar_Forceps*, *Prograsp_Forceps*, *Large_Needle_Driver*... Svaka klasa, osim *Other* i *Probe*, imaju dodatne klase komponente instrumenta za *Shaft*, *Wrist*, *Claspers*. Drugim riječima, kada ako slika sadrži *Bipolar_Forceps_Shaft*, ta klasa bit će zapisana 0, *Bipolar_Forceps_Wrist* 1 itd. Ostavljeno je nekoliko klasa za klasu *Other* u slučajevima ako postoji detektirani predmet, ali je odrezan na slici u nekoliko dijelova radi instrumenta.

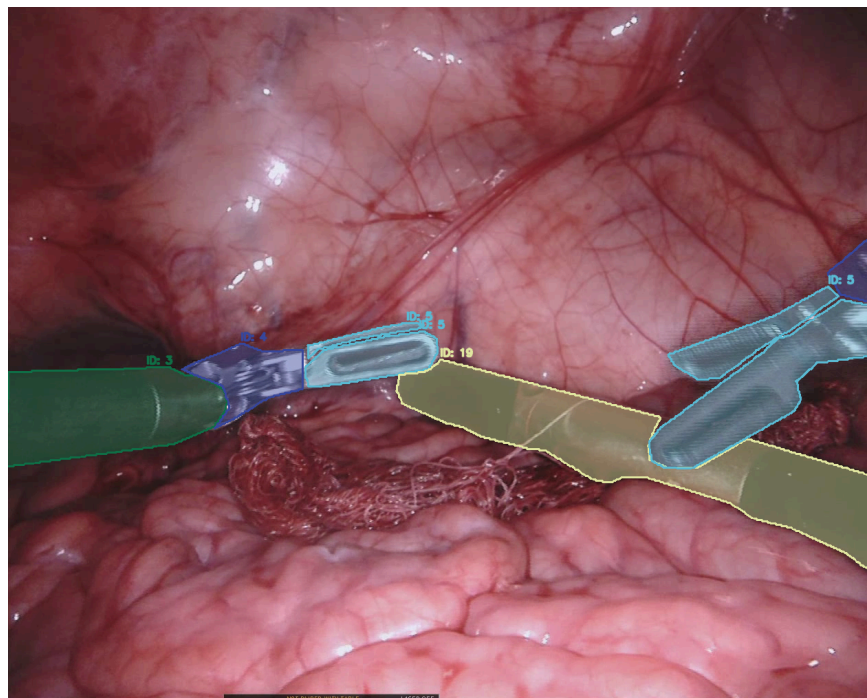
Idući dio skripte fokusira se na rezanje crnih pravokutnika na označenim slikama, bez spremanja tih slika. Te posljednji dio zapisuje tekstualnu datoteku gdje prva znamenka označava klasu, a druga dva broja su par koji označavaju lokaciju točke poligona po relativnoj dužini i

visini slike, ti parovi se ponavljaju sve dok se poligon nije sasvim opisao. Primjer tekstualnog zapisa oznake može biti pronađen na slici 3.6.

```
1 3 0.167187 0.521484 0.166406 0.522461 0.141406 0.522461 0.140625 0.523438
2 4 0.273438 0.487305 0.272656 0.488281 0.271875 0.488281 0.271094 0.489258
3 5 0.476562 0.470703 0.475781 0.471680 0.471094 0.471680 0.470313 0.472656
4 5 0.461719 0.458008 0.460938 0.458984 0.457031 0.458984 0.456250 0.459961
5 19 0.500781 0.510742 0.496875 0.515625 0.496094 0.515625 0.491406 0.521484
6 4 0.987500 0.336914 0.983594 0.341797 0.982812 0.341797 0.978125 0.347656
7 5 0.940625 0.406250 0.939844 0.407227 0.939063 0.407227 0.938281 0.408203
```

slika 3.6. zapis oznake prema uvjetu YOLOv8-seg modela

Zapis oznake je isti kao i prijašnji primjer, a vizualizacija takvog zapisa na primjeru slike može biti pronađen na slici 3.7.



slika 3.7. slika nakon primjene maske s oznakama

Skup koji je predviđen za testiranje nema iste oznake kao skup za treniranje, jer na slikama nedostaju oznake za instrumente poput instrumenta označenog brojem 19 na slici 3.7. Nadalje, u oznakama skupa za testiranje nedostaje klasifikacija instrumenta, ali postoje oznake za dijelove instrumenta, koji su, također, označeni drugačijim intenzitetom od onih u skupu za testiranje.

Augmentacije podataka koja je prisutna za poboljšanje rezultata modela su prikazane kao parametri modela, koji uključuju:

- *mosaic* - spaja 4 različite slike u jedan rešetkasti raspored za svaku iteraciju, pomaže učenju modelu za različite tipove objekata s raznim skalama i složenijim kontekstom u jednoj slici. Vrijednost parametra je 1,0.
- *mixup* - miješanje dvije različite slike i njihovih oznaka s faktorom prozirnosti kako bi model poboljšao glatkoću granica instrumenata. Vrijednost parametra je 0,2
- *copy_paste* - razina zaljepljivanja dijela objekta s jedna na drugu sliku, s vjerojatnošću od 30% u ovom radu, kako bi model poboljšao učenje na gustom rasporedu objekta
- *degrees* - vrijednost nasumičnog rotiranja slike, u ovom radu, do 20 stupnjeva u jedno od dva kuta.
- *flip_lr* - vrijednost koja predstavlja vjerojatnost, u ovom radu 50%, horizontalnog okretanja slike.
- *flipud* - vrijednost koja predstavlja vjerojatnost, u ovom radu 20%, vertikalnog okretanja slike.
- *hsv_s* - nasumično povećanje zasićenja slike, u ovom radu do 70%, kako bi se model prilagodio na okoline drugačijeg osvjetljenja.
- *hsv_v* - nasumično povećanje svjetline slike, u ovom radu do 40%, kako bi se model prilagodio na razne količine svjetlosti.

4. IMPLEMENTACIJA I TRENIRANJE YOLOv8 MODELA

Model se temelji na korištenje bilbioteke *ultralytics*, a specifična inačica korištenog modela u ovom radu je YOLOv8l-seg. Koristi se ubrzanje putem grafičke kartice, a ovisno o grafičkoj kartici, ako se koriste kartice AMD (engl. *Advanced Micro Devices*) arhitekture, sigurnije je izbjeći korištenje AMP-a (engl. *Automatic Mixed Precision*), koji ponekad može biti uzrok rušenja sustava.

Hiperparametri modela koji nisu mijenjani su standardne vrijednosti, a oni koji su mijenjani su:

- ☐ broj epoha - određuje koliko puta će model proći kroz cijeli skupo podataka tijekom treniranja, a proći će ga 50 puta.
- ☐ *batch size* - određuje broj slika koje se paralelno obrađuju te treba, s obzirom na složenost modela, paziti na iskorištenost VRAM-a (engl. *Video Random Access Memory*). U slučaju ovog rada, paralelno se obrađuju 2 slike.
- ☐ veličina slike - određuje veličinu ulaza, a uvjet modela je djeljivost slike s 32. Iako se postavila veličina na [640,968], model je promijenio veličinu slike na 992.
- ☐ stopa učenja - određuje koliko model mijenja parametre pri koraku optimizacijskog algoritma, a u radu se koristi stopa od 0,01.

S obzirom na oznake u skupu namijenjen za testiranje, koristit će se samo skup namijenjen za treniranje jer se želi postići prepoznavanje specifičnog instrumenata, a ne samo detekcija instrumenta. Koristi se 2 pristupa implementacije, jedan pristup uzima cijeli videozapis, a drugi uzima slike bez obzira jesu li iz istog videozapisa. Oba pristupa koriste *k-fold cross validation* koji dijeli skup podataka na 4 jednaka dijela, u slučaju uzimanja cijelog videozapisa, gdje se kao skup za testiranje koristi 2 videozapisa, a ostali su namijenjeni za skup za treniranje, dok drugi pristup dijeli skup na 5 jednakih dijelova, 360 slika po dijelu. Potrebno je obratiti pažnju u podijeli jer se kirurški zahvat dijeli na korake, a određeni koraci imaju, odnosno nemaju, iste intrumente, stoga u slučaju videzapisa potrebno je uzeti videozapise koji nisu slijedni.

Model koristi *yaml* datoteke koje određuju koliko klasa se koristi, u ovom slučaju je 23:

- ☐ *Bipolar_Forceps*
- ☐ *Prograsp_Forceps*
- ☐ *Large_Needle_Driver*

- ☐ *Vessel_Sealer*
- ☐ *Grasping_Retractor*
- ☐ *Monopolar_Curved_Scissors*
- ☐ *Probe*
- ☐ *Other*

Svaka klasa ima dodatak *_Shaft*, *_Wrist*, *_Claspers*, osim *Probe* i *Other*, a *Other* klasa ima pet ponavljanja jer model smatra koruptiranom datotekom oznaka, ako postoji više oznaka u tekstualnoj datoteci od *yaml* datoteke, a skripta za ekstrakciju oznaka smatra svaki instrument koji je zaklonjen kao dva odvojena instrumenta. *Yaml* datoteka usmjerava model na tekstualne datoteke koje sadrže apsolutne putanje slika koje se koristi za treniranje, odnosno validaciju. Postoji onoliko *yaml* datoteka koliko postoji i *foldova*.

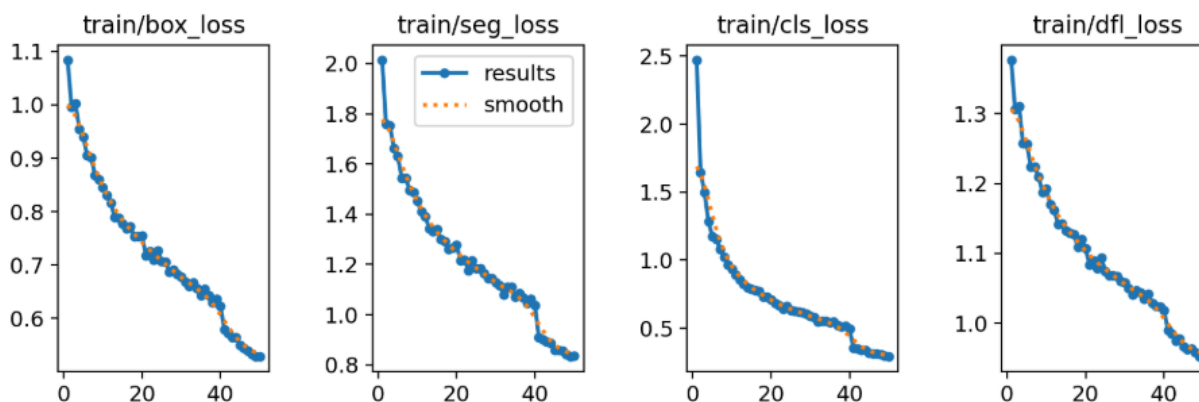
Dodatne metode sprječava *overfittinga* korištene u radu su:

- *dropout* - nasumična deaktivacija neurona tijekom procesa treniranja kako bi se spriječilo preveliko učenje pojedinih neurona, koristi se 0,15 ili 15% neurona.
- *weight decay* - primjenjuje kazna L2 regularizaciji kako model ne bi dodao prevelike vrijednosti svojim težinama, smanjuje se složenost i povećava generalizacija, a koristi se vrijednost od 0,001.
- *label smoothing* - smanjuje se apsolutnost klasa u oznakama gdje je model manje kažnjen za ne maksimalne vjerojatnosti, a u radu se koristi kao iznos od 0,1.

Rano zaustavljanje nije nužno koristiti kako bi se spriječio *overfitting* jer se po standardnim postavkama modela sprema najbolji model, kao i posljednji model.

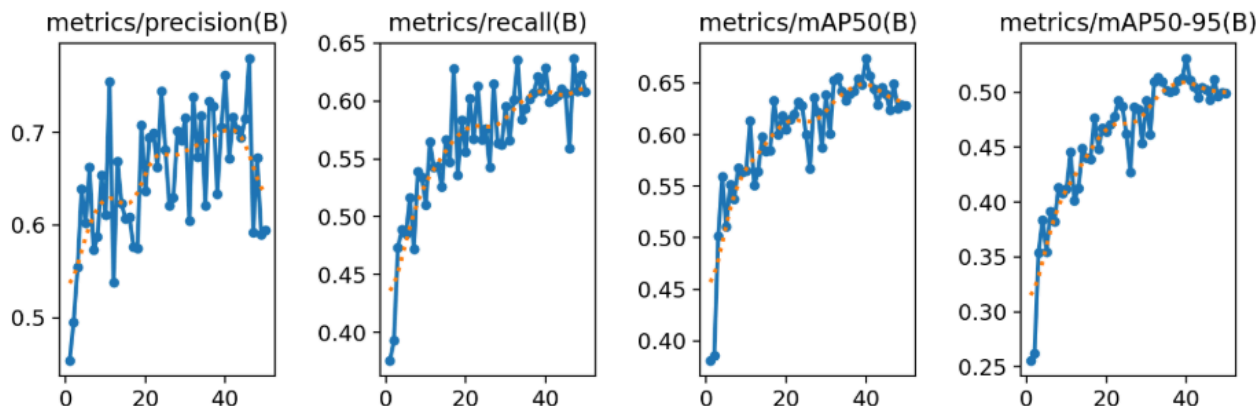
5. EVALUACIJA REZULTATA

Budući kako se radilo s 4 *foldova*, postoje razlike između njih jer drugi instrumenti sudjeluju u različitim koracima operacije, ali postoje i sličnosti. Za sve *foldove* gubitak tijekom treniranja izgleda kao na slici 5.1., gdje se može vidjeti kako se gubitak smanjuje.



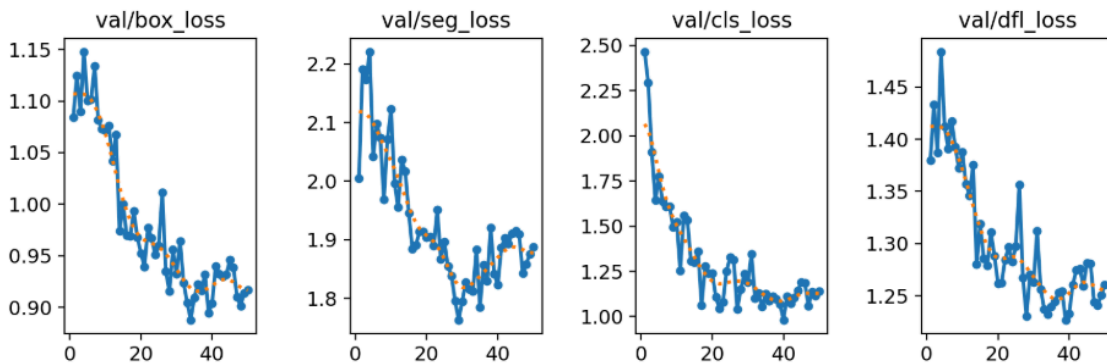
slika 5.1. grafovi gubitaka na skupu za treniranje prilikom treniranja prvog *folda*

Metrike prikazane na grafovima na slici 5.2. slijede relativno sličnu putanju među *foldovima*, razlike između grafova između *foldova* se može pronaći u vrijednosti, ali se za svaki *fold* povećavaju metrike.



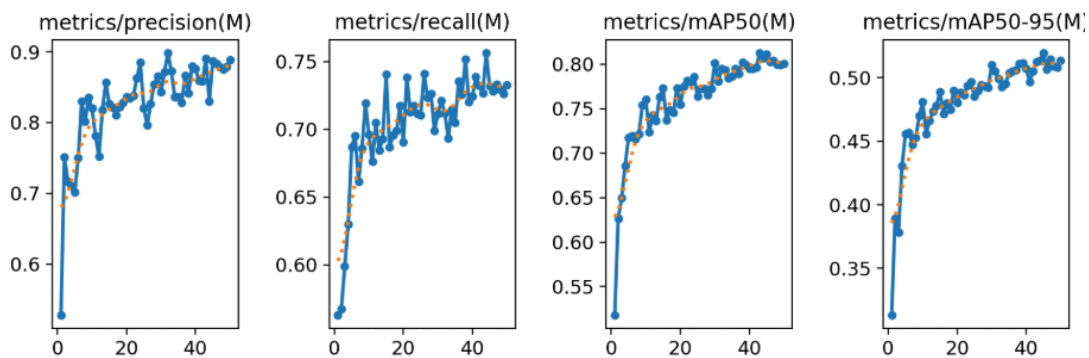
slika 5.2. grafovi metrika graničnih okvira prilikom treniranja prvog *folda*

Rezultati za validacijski dio se razlikuju između *foldova* jer se gubitak u prvom *foldu* smanjuje, osim kod segmentacijskog gubitka jer pokazuje znakove prevelikog prilagođavanja podacima tijekom 30-te epohe, prikazano na slici 5.3. Osim toga, *fold 2* pokazuje rast te metrike već u petoj epohi, što još više pokazuje na pogreške prilikom validacije.

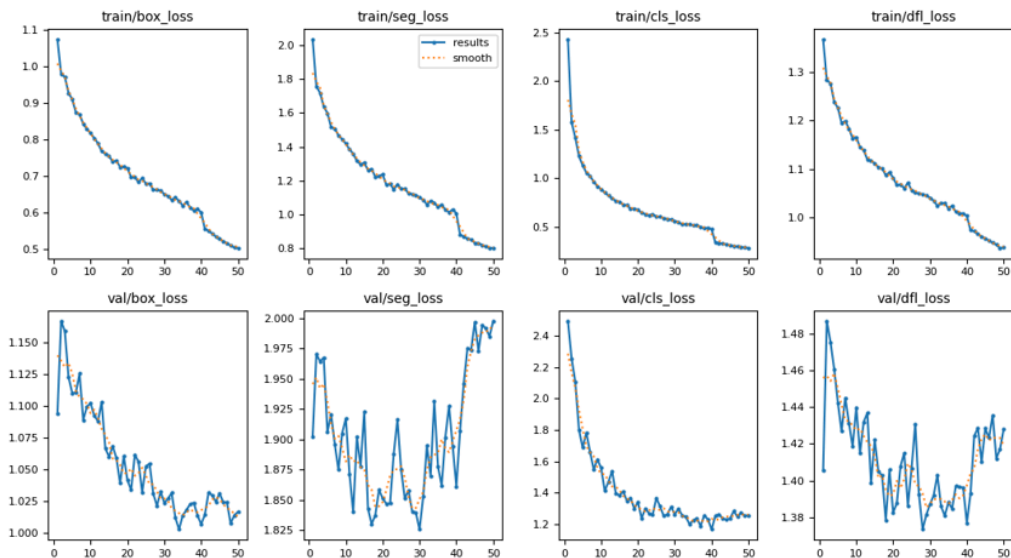


slika 5.3. grafovi gubitaka na validacijskom skupu prilikom treniranja prvog *folda*

Posljednji graf označava metrike maske na prvom *foldu* koje su prikazane na slici 5.4. One pokazuju isti trend na svim *foldovima*.

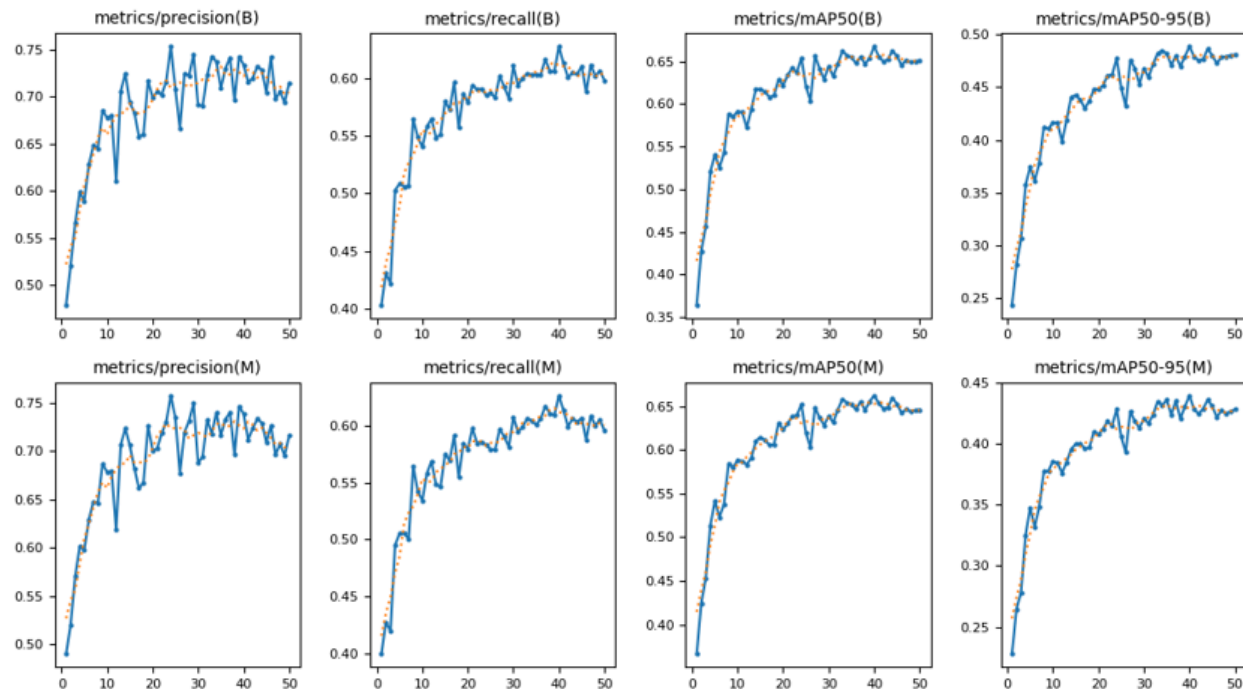


slika 5.4. grafovi metrike na maske prilikom treniranja prvog *folda*



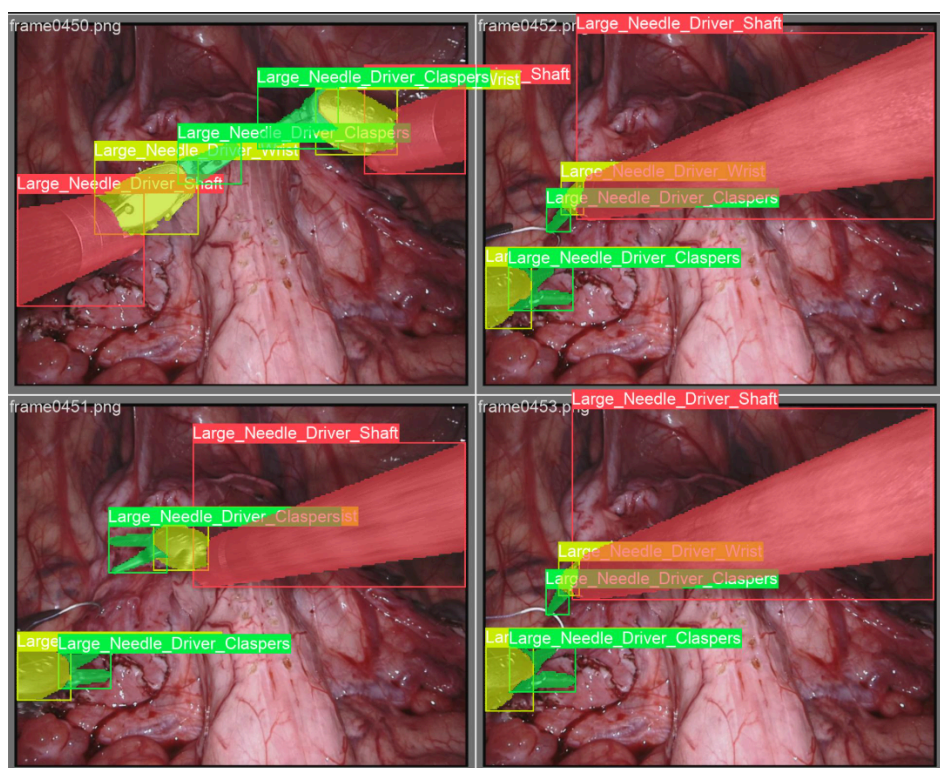
slika 5.5. grafovi gubitaka između svih *foldova*

Na slici 5.5, može se vidjeti kombinacija rezultata gubitaka svih *foldova* tijekom treniranja. Može se primijetiti kako poslije 30-te epohe kod validacijskog gubitka za segmentaciju i *distribution focal loss*, gubitci rastu, što je nepoželjno. Dok se na slici 5.6., mogu vidjeti metrike za sve *foldove*, koje gotovo uvijek pokazuju rast, ali vrijednost nije toliko visoka.

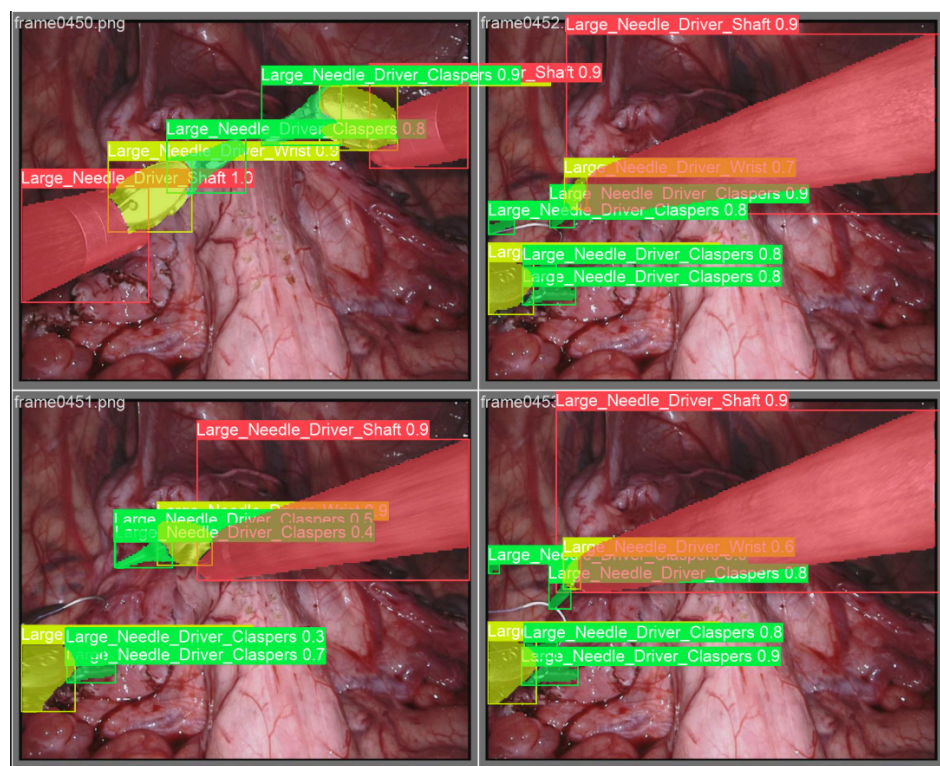


slika 5.6. grafovi metrika između svih *foldova*

Kao primjer na odabranim slikama iz videozapisa, na slikama 5.7. i 5.8. mogu se vidjeti upravo odabrane slike iz validacijskog skupa, gdje se na slici 5.7. prikazuju slike koje su točno označene, dok se na slikama na slici 5.8. može vidjeti predikcija modela istih slika. Mogu se vidjeti većinom točne predikcije, ali isto tako je zamijenio tkivo za instrument na slika 452 i 453.



slika 5.7. točno označene odabrane slike validacijskog skupa



slika 5.8. predikcija modela odabranih slika validacijskog skupa

6. RASPRAVA I ZAKLJUČAK

Zaključno, ovaj rad ukratko ulazi u problematiku detekcije objekata i navodi razlike između one-stage i two-stage detektora. Ulazi u razvoj one-stage detektora, specifično YOLO modela, od YOLOv1 do YOLOv8. Fokus primjene modela se stavlja u prepoznavanje kirurških instrumenata na intraoperativnim videozapisima gdje se koristi skup podataka s tom tematikom koji se obrađuje kao priprema za primjenu učenja YOLO modela nad njima. Objašnjavaju se tehnike predobrade podataka za YOLO nad skupom podataka te se implementiraju i trenira model s istim.

Određeni dijelova videozapisa imaju veliku količinu varijacije instrumenata jer je obični položaj instrumenata relativno udaljen od kamere, ali tijekom početka ili kraja uporabe instrumenta, taj instrument pokazuje veliko izobličenje. Također, može se dogoditi kako je instrument prevelik stoga veliki dijelovi instrumenta izlaze i ulaze u sliku kamere. Nadalje, postoji problem blokiranja instrumenta ili s drugim instrumentima ili sa tkivom tijekom operacije, stoga se još manje može prepoznati instrumenti ili se dijelovi instrumenta dijele na više instrumenata.

Dobiveni rezultati imaju problem sa prekomjernim prilagođavanjem podacima tj. *overfittingom* radi manjka podataka, a rezultati su mogli biti bolji kada bi se koristila vremenska sekvenca jer, u određenim dijelovima videozapisa, teško je odrediti instrument u tom trenutku, dok se jasnije može identificirati instrument u kasnijem ili ranijem dijelu videozapisa.

Osik toga, poboljšanje rezultata moguće je dobiti kada bi skup podataka imao više videozapisa, pogotovo jer se u ovom skupu, u slučaju nekoliko videozapisa koristi samo jedan instrument tijekom videozapisa. Također, rezultati bi se poboljšali kada bi označene slike imale bolje oznake i kada bi se te oznake bolje prenijele na zapis prikladan YOLOv8-seg modelu.

LITERATURA

- [1] J. R. Jeganathan, R. Jegasothy, and W. T. Sia, "Minimally invasive surgery: a historical and legal perspective on technological transformation," *Journal of Robotic Surgery*, vol. 19, no. 1, p. 408, 2025. [Online]. Available: <https://pmc.ncbi.nlm.nih.gov/articles/PMC12279690/>
- [2] R. Hargest, "Five thousand years of minimal access surgery: 3000BC to 1850: early instruments for viewing body cavities," *Journal of the Royal Society of Medicine*, vol. 113, no. 12, pp. 491–496, 2020. [Online]. Available: <https://pmc.ncbi.nlm.nih.gov/articles/PMC7816654/>
- [3] "History of Surgery - A Timeline," *Encyclopædia Britannica*. [Online]. Available: <https://www.britannica.com/science/History-of-Surgery-A-Timeline>
- [4] "Object Recognition and Selective Search," *MIT Vision Book*. [Online]. Available: https://visionbook.mit.edu/object_recognition_v3.html
- [5] "Object Detection Guide," *Lightly AI Blog*. [Online]. Available: <https://www.lightly.ai/blog/object-detection>
- [6] Z. Zou, K. Chen, Y. Shi, Y. Guo, and J. Ye, "A Survey of Deep Learning-Based Object Detection Methods and Datasets for Overhead Imagery," *arXiv preprint arXiv:2104.07876*, 2022. [Online]. Available: https://www.researchgate.net/publication/358362847_A_Survey_of_Deep_Learning-Based_Object_Detection_Methods_and_Datasets_for_Overhead_Imagery
- [7] S. Satyanarayana, "Selective Search for Object Detection," *LearnOpenCV*. [Online]. Available: <https://learnopencv.com/selective-search-for-object-detection-cpp-python/>
- [8] J. R. R. Uijlings, K. E. A. van de Sande, T. Gevers, and A. W. M. Smeulders, "Selective Search for Object Recognition," *Draft Publication*, 2013. [Online]. Available: <http://www.huppelen.nl/publications/selectiveSearchDraft.pdf>
- [9] R. Girshick, J. Donahue, T. Darrell, and J. Malik, "Rich feature hierarchies for accurate object detection and semantic segmentation," in *Proceedings of the IEEE Conference on*

Computer Vision and Pattern Recognition (CVPR), 2014, pp. 580–587. [Online]. Available: <https://arxiv.org/pdf/1311.2524>

[10] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 779–788. [Online]. Available: https://www.cv-foundation.org/openaccess/content_cvpr_2016/papers/Redmon_You_Only_Look_CVPR_2016_paper.pdf

[11] N. Jegham, C. Y. Koh, M. Abdelatti, and A. Hendawi, "YOLO Evolution: A Comprehensive Benchmark and Architectural Review of YOLOv12, YOLOv11, and Their Previous Versions," *arXiv preprint arXiv:2411.00201*, 2025. [Online]. Available: <https://arxiv.org/html/2411.00201v2>

[12] J. Terven and D. Cordova-Esparza, "A Comprehensive Review of YOLO Architectures in Computer Vision: From YOLOv1 to YOLOv8 and YOLO-NAS," *Machine Learning and Knowledge Extraction*, vol. 5, no. 4, pp. 1680–1716, 2023. [Online]. Available: <https://arxiv.org/pdf/2304.00501>

[13] J. Redmon and A. Farhadi, "YOLO9000: Better, Faster, Stronger," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 7263–7271. [Online]. Available: <https://arxiv.org/pdf/1612.08242>

[14] J. Redmon and A. Farhadi, "YOLOv3: An Incremental Improvement," *arXiv preprint arXiv:1804.02767*, 2018. [Online]. Available: <https://arxiv.org/pdf/1804.02767>

[15] A. Bochkovskiy, C.-Y. Wang, and H.-Y. M. Liao, "YOLOv4: Optimal Speed and Accuracy of Object Detection," *arXiv preprint arXiv:2004.1034*, 2020. [Online]. Available: <https://arxiv.org/pdf/2004.10934>

[16] "YOLOv5 Architecture Description," *Ultralytics Docs*. [Online]. Available: <https://docs.ultralytics.com/yolov5/tutorials/architecture-description>

[17] C. Li *et al.*, "YOLOv6: A Single-Stage Object Detection Framework for Industrial Applications," *arXiv preprint arXiv:2301.05586*, 2023. [Online]. Available: <https://arxiv.org/pdf/2301.05586>

- [18] C.-Y. Wang, A. Bochkovskiy, and H.-Y. M. Liao, "YOLOv7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023, pp. 7464–7475. [Online]. Available: <https://arxiv.org/pdf/2207.02696>
- [19] "YOLOv8 Model Documentation," *Ultralytics GitHub Repository*. [Online]. Available: <https://github.com/ultralytics/ultralytics/blob/main/docs/en/models/yolov8.md>
- [20] "Ultralytics YOLO Documentation," *Ultralytics Docs*. [Online]. Available: <https://docs.ultralytics.com/>